

République du Cameroun
Paix – Travail – Patrie

Université de Yaoundé I

École Nationale Supérieure Polytechnique
de Yaoundé
(ENSPY)

Département de Génie Informatique



Republic of Cameroon
Peace – Work – Fatherland

University of Yaoundé I

National Advanced School of Engineering
of Yaoundé
(NASEY)

Department of Computer Engineering

Rapport de projet

Business Core as a Service

*Une infrastructure générique de gestion des métiers
inspirée de l'ingénierie des protocoles réseaux*

Groupe 3

Nom & Prénom(s)	Matricule	Participation (%)
ETCHOME Chris Manuel Loïc	23P393	100
KEMDA DEMANOU Jean Loïc	22P356	100
KENNE DIHA Ange Léa	23P566	70
KONGNI TCHINDA Jude Roland	23P349	70
KOTCHAP TINGA Blessing	23P350	60
MBA BELA Océanne	23P518	70

Sous la supervision de :
Pr. Dr Ing. **Thomas Djotio Ndié**

Assistant :
Juslin KUTCHE NEGOUE
MSc, PhD Student

Résumé

Ce rapport présente la conception, l'implémentation et le déploiement de **BCaaS** (*Business Core as a Service*), une plateforme générique *multi-tenant* dédiée à la gestion standardisée des processus métier. Le projet s'inscrit dans le cadre du cours d'Introduction aux Réseaux de la troisième année de Génie Informatique de l'École Nationale Supérieure Polytechnique de Yaoundé. L'idée centrale consiste à modéliser un système d'information métier en s'appuyant rigoureusement sur les concepts de l'ingénierie des protocoles réseaux : pile de couches, encapsulation, routage, adressage, qualité de service, et séparation des plans de contrôle et de données. Cette analogie formelle, loin d'être une simple métaphore pédagogique, structure l'ensemble des décisions architecturales : un *tenant* y joue le rôle d'un VLAN, une demande de service celui d'un paquet, et la facture celui d'un accusé de réception. La solution livrée repose sur Spring Boot 3.5 / Java 21, PostgreSQL avec Liquibase, Spring Security avec authentification JWT et isolation multi-tenant par colonne `tenant_id`, et SpringDoc OpenAPI pour la documentation automatique de l'API. Le domaine compte quinze entités et l'API expose environ quatre-vingt-cinq points de terminaison REST validés par cent cinquante-deux tests automatisés. Un frontend Next.js (site vitrine et tableau de bord d'administration) complète l'ensemble. L'API est déployée sur Render et la documentation sur Vercel.

Mots-clés : Business Core, multi-tenant, protocoles réseaux, modèle OSI, encapsulation, Spring Boot, API REST, JWT, Next.js, architecture hexagonale.

Abstract

This report presents the design, implementation and deployment of **BCaaS** (*Business Core as a Service*), a generic *multi-tenant* platform dedicated to the standardized management of business processes. The project was carried out within the Introduction to Networks course of the third year of Computer Engineering at the National Advanced School of Engineering of Yaoundé. The core idea is to model a business information system by rigorously relying on the concepts of network protocol engineering: layered stack, encapsulation, routing, addressing, quality of service, and the separation of control and data planes. This formal analogy is not a mere pedagogical metaphor; it drives every architectural decision: a *tenant* plays the role of a VLAN, a service request that of a packet, and an invoice that of an acknowledgement signal. The delivered solution is built on Spring Boot 3.5 / Java 21, PostgreSQL with Liquibase, Spring Security with JWT authentication and `tenant_id`-based multi-tenant isolation, and SpringDoc OpenAPI for automatic API documentation. The domain comprises fifteen entities and the API exposes roughly eighty-five REST endpoints validated by one hundred and fifty-two automated tests. A Next.js frontend (public showcase site and administration dashboard) completes the system. The API is deployed on Render and the documentation on Vercel.

Keywords: Business Core, multi-tenant, network protocols, OSI model, encapsulation, Spring Boot, REST API, JWT, Next.js, hexagonal architecture.

Table des matières

Résumé	i
Abstract	ii
Liste des abréviations	vi
1 Introduction générale	1
1.1 Contexte du projet	1
1.2 Problème	1
1.3 Problématique	2
1.4 Objectifs	2
1.4.1 Objectif général	2
1.4.2 Objectifs spécifiques	2
1.5 Méthodologie	3
1.6 Structure du document	4
2 État de l’art	5
2.1 Modèles en couches : OSI et TCP/IP	5
2.1.1 Le modèle OSI	5
2.1.2 La suite TCP/IP	6
2.1.3 Intérêt méthodologique pour BCaaS	7
2.2 Architectures multi-tenant	7
2.2.1 Définition et enjeux	7
2.2.2 Trois stratégies d’isolation	8
2.2.3 Analogie avec les réseaux virtuels (VLAN)	9
2.3 Le paradigme <i>as-a-Service</i> et le <i>Business as a Service</i>	9
2.3.1 L’évolution des modèles <i>as-a-Service</i>	9
2.3.2 Le <i>Business as a Service</i> (BaaS)	10
2.3.3 Avantages et défis du BaaS	10
2.4 Architecture hexagonale et <i>Domain-Driven Design</i>	10
2.4.1 L’architecture hexagonale (<i>Ports & Adaptateurs</i>)	10
2.4.2 Le <i>Domain-Driven Design</i> (DDD)	11
2.4.3 DDD et architecture hexagonale : une synergie naturelle	12
2.5 Positionnement de BCaaS	12
2.5.1 Ce que la littérature fournit déjà	12
2.5.2 Le <i>gap</i> identifié	12
2.5.3 La contribution de BCaaS	13

Conclusion du chapitre	13
3 Analogie formelle entre réseaux et métier	14
3.1 Table de correspondance réseau–métier	14
3.2 Émetteur, récepteur, protocole et message	15
3.3 Flux de traitement d’une requête	16
3.4 La pile protocolaire métier en cinq couches	16
3.4.1 Couche 1 — Infrastructure	16
3.4.2 Couche 2 — Transport & Messaging	17
3.4.3 Couche 3 — Tenant & Routing	17
3.4.4 Couche 4 — Context & Policy	18
3.4.5 Couche 5 — Business Capabilities	18
3.5 Le multi-tenant comme réseau virtuel (VLAN)	18
3.6 Plan de contrôle et plan de données	18
3.7 Routage applicatif et recherche de chemin logique	19
3.8 Contrôle de flux et qualité de service (QoS)	19
4 Analyse et spécification des besoins	20
4.1 Besoins fonctionnels	20
4.2 Besoins non fonctionnels	21
4.3 Diagramme de contexte	22
5 Modélisation mathématique	23
5.1 Ensembles fondamentaux	23
5.1.1 Décomposition des acteurs	23
5.1.2 Attributs des entités principales	23
5.2 Variables de décision et fonction objectif	23
5.2.1 Variables	24
5.3 Contraintes du système	24
5.3.1 Contraintes de cardinalité	24
5.3.2 Contraintes métier	24
5.3.3 Contrainte de workflow : machine à états	24
5.4 Algorithmes et complexité	25
5.4.1 Recherche des fournisseurs d’un métier	25
5.4.2 Recherche par filtres combinés	25
5.5 Indicateurs et métriques (KPIs)	25
5.6 Architecture en couches : composition de fonctions	26
5.6.1 Définition formelle	26
5.6.2 Propriété : encapsulation stricte	26
5.6.3 Exemple	27
5.7 Isolation multi-tenant : théorie des ensembles	27
5.7.1 Définitions	27
5.7.2 Propriété : isolation totale	27

5.7.3	Exemple	28
5.8	Provisionnement d'un tenant : suite finie d'états	28
5.8.1	Les sept états du provisionnement	28
5.8.2	Propriété : injectivité du domaine	28
5.8.3	Exemple	28
6	Conception architecturale	30
6.1	Architecture hexagonale et DDD	30
6.2	Monolithe modulaire « microservices-ready »	30
6.3	Diagramme de packages	31
7	Conception détaillée des données	33
7.1	Dictionnaire des entités	33
7.2	Modèle Logique de Données (MLD)	34
7.3	Diagramme de classes métier	35
7.4	Contraintes d'intégrité	35
7.5	Machines à états	36
8	Implémentation technique	37
8.1	Pile technologique	37
8.2	Structure du projet	37
8.3	Multi-tenant et isolation	38
8.4	Sécurité : JWT et RBAC	39
8.5	Persistance et migrations	39
8.6	Temps réel et événementiel	39
8.7	Documentation et conteneurisation	39
9	Protocole d'échange pour un cas métier concret	40
9.1	Cas choisi : création d'un Business et ajout au catalogue	40
9.2	Protocole : création d'un Business	40
9.3	Protocole : ajout au catalogue de services	41
10	Traçabilité et observabilité	42
10.1	Piste d'audit	42
10.2	Tracing : <code>traceId</code> et <code>correlationId</code>	43
10.3	Analytique et KPIs	44
10.4	Logs structurés (proposition)	45
11	Frontend et déploiement	46
11.1	Architecture du frontend	46
11.2	Choix techniques	46
11.3	Pages principales	47
11.4	Déploiement	47

12 Tests et validation	50
12.1 Stratégie de test	50
12.1.1 Vue d'ensemble et outillage	50
12.1.2 Environnement de test	51
12.1.3 Les trois niveaux de test	51
12.2 Critères d'acceptation	53
12.2.1 Définition et méthode	53
12.2.2 Tableau des critères	53
12.2.3 Analyse des critères	54
12.3 Scénario de validation bout-en-bout	55
12.3.1 Objectif et portée	55
12.3.2 Déroulement du scénario nominal	55
12.3.3 Ce que le scénario valide	56
12.4 Note sur la couverture de code et perspectives	57
12.4.1 État actuel	57
12.4.2 Perspectives d'amélioration	57
Synthèse	57
13 Perspectives d'amélioration	59
13.1 Court terme : finalisation des briques posées	59
13.2 Moyen terme : robustesse et qualité de service	59
13.3 Long terme : montée en charge et nouvelles contraintes	60
14 Conclusion générale	61
14.1 Bilan	61
14.2 Limites	61
14.3 Apports personnels	62
Bibliographie	63
Glossaire	64
A Annexes	66
A.1 Liste des endpoints REST	66
A.2 Exemple de log structuré	67
A.3 Capture Swagger UI	67
A.4 Extraits de code significatifs	67

Table des figures

3.1	Flux bout-en-bout d'une requête dans BCaaS : Client → API Gateway → Tenant & Routing → Context & Policy → Business Service, avec correspondance OSI descendante/ascendante et publication d'événements (notification, audit). . . .	17
3.2	Pile protocolaire métier de BCaaS et correspondance avec le modèle OSI	17
4.1	Diagramme de contexte du système	22
6.1	Architecture logique de BCaaS	31
6.2	Diagramme de packages de l'architecture du système	32
7.1	Diagramme de classes métier de BCaaS	35
7.2	Machine à états de ServiceRequest	36
7.3	Machine à états de Invoice	36
11.1	Diagramme de déploiement du système	48
11.2	Page d'accueil du site vitrine déployé sur Vercel	48
11.3	Documentation interactive de l'API	49
A.1	Documentation interactive Swagger UI	67

Liste des tableaux

2.1	Les sept couches OSI et leurs analogies dans BCaaS	6
2.2	Comparaison des stratégies d'isolation multi-tenant	8
2.3	Comparaison des modèles <i>as-a-Service</i> et positionnement de BCaaS	9
3.1	Correspondance formelle entre concepts réseaux et concepts métier	15
4.1	Besoins fonctionnels — Identités (Tenant, User, Actor)	20
4.2	Besoins fonctionnels — Métiers (Business, Rule, Catalogue, Resource, Media, Portfolio)	20
4.3	Besoins fonctionnels — Transactions (ServiceRequest, Invoice, Message, Notification)	21
4.4	Besoins non fonctionnels et statut dans le MVP	21
5.1	Transitions valides de la machine à états ServiceRequest	25
5.2	Suite d'états du provisionnement d'un tenant	28
7.1	Entités du modèle de données — Business Core	33
7.2	Entités du modèle de données – Business Core	34
7.3	Principales contraintes d'intégrité	36
8.1	Pile technologique de BCaaS	37
10.1	Métriques de BCaaS et leur analogie réseau	44
11.1	Technologies du frontend et leur rôle architectural	46
12.1	Critères d'acceptation et méthodes de vérification	53
A.1	Synthèse des endpoints REST	66

Liste des abréviations

Sigle	Signification
BCaaS	Business Core as a Service
BaaS	Business as a Service
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
RBAC	Role-Based Access Control
DDD	Domain-Driven Design
DTO	Data Transfer Object
FSM	Finite State Machine (machine à états finis)
OSI	Open Systems Interconnection
TCP/IP	Transmission Control Protocol / Internet Protocol
VLAN	Virtual Local Area Network
SDN	Software-Defined Networking
QoS	Quality of Service
SLA	Service-Level Agreement
MCD	Modèle Conceptuel de Données
MLD	Modèle Logique de Données
UML	Unified Modeling Language
PWA	Progressive Web App
SSR	Server-Side Rendering
CdS	Consommateur de Service
FdS	Fournisseur de Service
KPI	Key Performance Indicator
RLS	Row-Level Security
UUID	Universally Unique Identifier

CHAPITRE 1

Introduction générale

1.1 Contexte du projet

L'application métier est par définition une application développée selon des besoins métier. Elle permet de faciliter la gestion des activités d'une entreprise en simplifiant les tâches tout en automatisant les process. Depuis l'essor de la digitalisation, télétravail et BYOD (Bring your own device), la tendance est aux applications développées selon les spécificités de chaque métier ou chaque profil. Ainsi dans un contexte où les organisations multiplient ces applications métier spécifiques à chaque secteur d'activité, chaque nouveau projet métier, qu'il s'agisse d'une pharmacie, d'une agence immobilière, d'un cabinet médical, d'une auto-école ou d'une école, elle reconstruit invariablement les mêmes fondations : gestion des acteurs, des ressources, des règles métier, des workflows, ou des mécanismes d'audit, les mêmes briques logicielles sont reconstruites à chaque nouveau projet.

Parallèlement, l'ingénierie des réseaux informatiques a depuis longtemps résolu un problème structurellement similaire : permettre à des flux de données hétérogènes de circuler de manière fiable, sécurisée et interopérable à travers une infrastructure partagée. Le modèle OSI et la suite TCP/IP proposent une décomposition en couches aux responsabilités clairement délimitées, un mécanisme d'encapsulation des données, un adressage précis des destinataires, une qualité de service différenciée, et une séparation nette entre plan de contrôle et plan de données. Les principes qui gouvernent les réseaux séparation des responsabilités en couches, encapsulation des données, routage, contrôle de la qualité de service constituent une référence éprouvée pour structurer des systèmes complexes. Les principes qui gouvernent les réseaux séparation des responsabilités en couches, encapsulation des données, routage, contrôle de la qualité de service constituent une référence éprouvée pour structurer des systèmes complexes. C'est dans ce contexte que s'inscrit le concept de Business Core as a Service (BCaaS) : une infrastructure générique de gestion des métiers inspirée de l'ingénierie des protocoles réseaux.

1.2 Problème

Les applications métier (gestion de pharmacie, d'agence immobilière, de cabinet médical, d'auto-école, d'école, etc.) reposent toutes sur les mêmes fondations : gestion des acteurs, des ressources, des règles métier, des workflows et de l'audit. Pourtant, ces briques communes sont reconstruites à chaque nouveau projet, sans méthode de conception réutilisable ni séparation claire des responsabilités, ce qui rend ces applications difficiles à faire évoluer et peu adaptées à une utilisation par plusieurs clients (tenants) sur une même infrastructure.

Le problème qui se pose est donc le suivant : concevoir une infrastructure métier générique, modulaire et multitenant, en s'appuyant sur les principes de structuration en couches, d'encapsulation et de routage déjà éprouvés par l'ingénierie des protocoles réseaux

1.3 Problématique

Question directrice

Comment concevoir un noyau métier générique, réutilisable et configurable qui permette à n'importe quelle instance métier de fonctionner sans reconstruire les fondations, en s'appuyant formellement sur les concepts de l'ingénierie des protocoles réseaux ?

Cette question directrice se décline en sous-questions à savoir :

Sur la structuration en couches: Comment décomposer un Business Core en couches aux responsabilités clairement délimitées, à l'image du modèle OSI, de sorte que chaque couche masque sa complexité aux couches voisines tout en leur offrant des services bien définis ?

Sur l'encapsulation des échanges: Comment séparer, au sein d'un même message métier, les métadonnées de routage et de contexte (identité du tenant, de l'acteur, traçabilité) du contenu métier proprement dit, afin qu'un service puisse appliquer les politiques communes sans connaître l'ensemble de l'application ?

Sur le routage et l'identification: Comment résoudre, pour chaque requête entrante, le tenant, le domaine métier et le service cible auquel elle doit être acheminée, de manière analogue à l'adressage IP dans un réseau ?

Sur l'isolation multitenant: Comment garantir qu'une infrastructure partagée entre plusieurs clients préserve une isolation stricte de leurs données et de leurs processus, à l'image de la virtualisation réseau (VLAN, namespace) ?

Sur la séparation gouvernance/exécution: Comment distinguer, au sein du Business Core, un plan de contrôle (configuration des règles, des tenants, des SLA) d'un plan de données (exécution effective des transactions métier), afin de permettre une reconfiguration du système sans interruption de service ?

1.4 Objectifs

1.4.1 Objectif général

Concevoir et réaliser une plateforme générique multi-tenant de gestion des métiers, architecturée selon une analogie rigoureuse avec la pile protocolaire réseau.

1.4.2 Objectifs spécifiques

1. Établir une analogie formelle et rigoureuse entre les concepts réseaux (couches OSI, encapsulation, routage, QoS, multi-tenant) et les composants d'une architecture métier

générique.

2. Définir une pile protocolaire métier en cinq couches (infrastructure, transport et messaging, tenant et routage, contexte et politique, capacités métier), où chaque couche assure une fonction précise et masque sa complexité aux couches voisines.
3. Implémenter une API REST de provisionnement de tenants reposant sur un modèle de données générique configurable.
4. Proposer une stratégie de multitenant et d'isolation garantissant la séparation logique des données et des processus entre les différents clients partageant la même infrastructure.
5. Documenter et exposer l'API via Swagger UI, et déployer la solution (API et documentation) sur le cloud.
6. Développer un frontend Next.js comprenant un site vitrine et la documentation détaillée de l'API REST.

1.5 Méthodologie

Pour répondre aux objectifs fixés, notre démarche s'appuie sur une progression en quatre étapes successives.

La première étape consiste à établir une analyse comparative entre les concepts fondamentaux de l'ingénierie des réseaux (modèle OSI, adressage IP, encapsulation, contrôle de la qualité de service) et les besoins propres à une architecture métier générique. Cette étape pose les bases conceptuelles du projet en faisant correspondre chaque mécanisme réseau à un équivalent métier.

La deuxième étape traduit cette analyse en un modèle mathématique d'une part et architectural d'autre part, structuré selon une pile de cinq couches respectant les principes d'abstraction et d'encapsulation. Chaque couche est définie avec ses responsabilités propres et ses interfaces vis-à-vis des couches voisines.

La troisième étape regroupe la conception, l'implémentation et les tests du système. Elle consiste d'abord à appliquer l'architecture définie à un scénario métier précis (santé, transport, logistique, etc.), en modélisant un flux d'échange complet, de la requête initiale à la propagation des événements. Cette conception est ensuite mise en œuvre à travers le développement des composants du Business Core, incluant la stratégie d'isolation multitenant garantissant la séparation des données entre clients. Enfin, des tests sont menés pour valider le bon fonctionnement de l'architecture, vérifier la cohérence des contrats d'échange entre services et s'assurer du respect des exigences de sécurité et d'isolation.

La quatrième étape ouvre une réflexion sur les perspectives d'évolution du Business Core, à travers la mise en place de mécanismes d'observabilité et de qualité de service, ainsi que l'identi-

fication des pistes d’extension possibles : ajout de nouveaux domaines métier, enrichissement des règles de gouvernance, ou amélioration de la résilience du système face à la montée en charge.

1.6 Structure du document

Le rapport est organisé comme suit. Le chapitre 2 dresse l’état de l’art. Le chapitre 3 formalise l’analogie réseau-métier, cœur conceptuel du projet, et y présente la pile protocolaire en cinq couches. Le chapitre 4 spécifie les besoins fonctionnels et non fonctionnels. Le chapitre 5 propose une modélisation mathématique du système. Les chapitres 6 et 7 détaillent la conception architecturale et la conception des données. Le chapitre 8 décrit l’implémentation technique, le chapitre 9 le protocole d’échange d’un cas métier, et le chapitre 10 la traçabilité et l’observabilité. Le chapitre 11 traite du frontend et du déploiement, le chapitre ?? des tests et de la validation, et le chapitre ?? de la gestion de projet. Le chapitre 13 expose les perspectives d’amélioration, avant la conclusion générale (chapitre 14).

CHAPITRE 2

État de l'art

Ce chapitre situe le projet BCaaS (*Business Core as a Service*) dans son paysage scientifique et technique. Il ne s'agit pas simplement de lister des concepts existants, mais de montrer comment chacun d'eux contribue à la conception de BCaaS et en quoi leur combinaison constitue une approche originale. Nous passerons successivement en revue les modèles en couches issus des réseaux informatiques, les stratégies d'architecture multi-tenant, le paradigme *as-a-Service* et son extension au niveau métier, les principes de l'architecture hexagonale et du *Domain-Driven Design*, avant de conclure en positionnant la contribution spécifique de BCaaS par rapport à cet état de l'art.

2.1 Modèles en couches : OSI et TCP/IP

2.1.1 Le modèle OSI

Le modèle OSI (*Open Systems Interconnection*), normalisé par l'ISO sous la référence ISO/IEC 7498-1 [2], est un cadre de référence conceptuel qui décompose la communication entre deux systèmes informatiques en sept couches aux responsabilités strictement délimitées. Chaque couche remplit une fonction précise et ne communique directement qu'avec les couches immédiatement adjacentes, sans connaître les détails de fonctionnement des autres.

Les sept couches, de la plus basse à la plus haute, sont les suivantes :

Couche 1 Physique

Transmission des bits bruts sur le médium (câble, fibre, ondes radio). Elle définit les niveaux de tension, les fréquences et les connecteurs physiques.

Couche 2 Liaison

Organisation des bits en trames, détection (voire correction) d'erreurs de transmission, et contrôle d'accès au médium partagé (protocoles Ethernet, Wi-Fi).

Couche 3 Réseau

Adressage logique (adresses IP) et routage des paquets entre réseaux différents. C'est la couche qui permet à un paquet de traverser plusieurs réseaux intermédiaires pour atteindre sa destination.

Couche 4 Transport

Communication de bout en bout entre processus, avec ou sans garantie de fiabilité. TCP assure la livraison ordonnée et sans perte ; UDP privilégie la rapidité au détriment de cette garantie.

Couche 5 Session

Établissement, maintien et terminaison des sessions de dialogue entre deux applications.

Couche 6 Présentation

Mise en forme des données (encodage des caractères, compression, chiffrement) afin que les applications puissent se comprendre indépendamment de leurs représentations internes.

Couche 7 Application

Interface directe avec les logiciels utilisateurs. C'est ici que résident les protocoles applicatifs tels que HTTP, SMTP, FTP ou DNS.

Le principe central du modèle OSI est l'**encapsulation** : lorsqu'une donnée descend la pile d'émission, chaque couche y ajoute un en-tête (et parfois un pied de trame) contenant les informations de contrôle nécessaires à son homologue sur la machine distante. À réception, la pile est parcourue en sens inverse : chaque couche lit et retire son en-tête avant de passer les données à la couche supérieure. Ce mécanisme garantit une séparation complète des responsabilités : chaque couche ignore le contenu des données qu'elle transporte et ne se préoccupe que de sa propre fonction.

Table 2.1: Les sept couches OSI et leurs analogies dans BCaaS

N°	Couche	Rôle principal	Analogie BCaaS
7	Application	Interface avec l'utilisateur final	API métier exposée aux clients
6	Présentation	Encodage, chiffrement, compression	Sérialisation JSON/XML, tokens JWT
5	Session	Gestion des sessions de communication	Gestion des sessions utilisateur
4	Transport	Fiabilité, contrôle de flux (TCP/UDP)	File d'attente, retry, transactions
3	Réseau	Adressage et routage inter-réseaux	Routage multi-tenant, isolation VLAN
2	Liaison	Détection d'erreurs, accès au médium	Validation des données, contraintes DB
1	Physique	Transmission binaire sur le médium	Infrastructure serveur, cloud

2.1.2 La suite TCP/IP

La suite de protocoles TCP/IP, développée par le Département de la Défense américain dans les années 1970 et standardisée au sein des RFC (*Request For Comments*), adopte une approche plus pragmatique en condensant les sept couches OSI en quatre niveaux : accès réseau (couches 1-2), Internet (couche 3), transport (couche 4) et application (couches 5-7). C'est ce modèle qui régit le fonctionnement d'Internet depuis son origine.

TCP/IP conserve les mêmes idées directrices que le modèle OSI — adressage, routage, fiabilité, contrôle de flux — mais en simplifiant leur mise en œuvre. Son adoption universelle a été rendue possible par sa flexibilité : l'adressage IP, notamment avec IPv6, permet d'identifier de manière unique chaque appareil connecté à l'échelle mondiale.

2.1.3 Intérêt méthodologique pour BCaaS

Au-delà de leur application aux réseaux informatiques, ces modèles présentent un intérêt méthodologique fondamental pour la conception logicielle. Trois principes sont particulièrement transposables :

Séparation des préoccupations par couches (*Separation of Concerns*) Chaque module ou composant logiciel est responsable d'une et d'une seule fonction. Ce principe réduit la complexité, facilite les tests unitaires et rend les composants remplaçables indépendamment.

Encapsulation progressive du contexte Chaque couche enrichit les données d'informations supplémentaires (métadonnées, identifiants, contexte de sécurité) sans que les couches adjacentes n'aient à connaître ces détails. En logiciel, cela correspond à l'enrichissement d'un objet métier à mesure qu'il traverse les différentes couches d'une application (validation, transformation, persistance).

Adressage explicite des destinataires Dans TCP/IP, une adresse IP et un port identifient sans ambiguïté la destination d'un paquet. En architecture multi-tenant, l'identifiant du tenant (`tenant_id`) joue un rôle analogue : il adresse chaque requête vers le bon contexte de données et de règles métier.

C'est cette transposition que BCaaS exploite explicitement : la pile applicative métier est conçue comme une pile protocolaire, avec des couches aux responsabilités distinctes, des mécanismes d'encapsulation du contexte (tenant, utilisateur, droits) et un adressage explicite par `tenant_id`.

2.2 Architectures multi-tenant

2.2.1 Définition et enjeux

Une application est dite **multi-tenant** (multi-locataire) lorsqu'une seule instance logicielle sert simultanément plusieurs organisations ou clients distincts (les *tenants*) tout en garantissant l'isolation totale de leurs données et configurations. Ce modèle s'oppose au modèle multi-instance, où chaque client dispose de sa propre installation du logiciel [13].

Les enjeux du multi-tenant sont multiples et parfois contradictoires :

Isolation des données Un tenant ne doit jamais pouvoir accéder, même accidentellement, aux données d'un autre. Cette exigence est à la fois technique (partitionnement physique ou logique) et réglementaire (RGPD, normes sectorielles).

Performance et scalabilité Le partage d'infrastructure peut introduire des effets de voisinage (*noisy neighbor problem*) où un tenant très actif dégrade les performances des autres.

Personnalisation Chaque tenant peut avoir des configurations, des règles métier ou des interfaces différentes, tout en partageant le même code de base.

Opérabilité Les mises à jour, les sauvegardes et la supervision doivent être gérées de manière centralisée pour tous les tenants, sans interruption de service.

2.2.2 Trois stratégies d'isolation

La littérature académique et les pratiques de l'industrie distinguent classiquement trois niveaux d'isolation, chacun avec ses compromis en termes de sécurité, de performance et de coût opérationnel [7].

Table 2.2: Comparaison des stratégies d'isolation multi-tenant

Stratégie	Isolation	Performance	Complexité / Coût
Colonne <code>tenant_id</code> (schéma partagé)	Logique	Bonne (index)	Faible idéal pour un MVP
Schéma PostgreSQL séparé	Physique	Très bonne	Moyenne
Base de données séparée	Totale	Variable	Élevée

a) Schéma partagé avec colonne `tenant_id`

C'est la stratégie la plus simple. Toutes les données de tous les tenants coexistent dans les mêmes tables. Une colonne `tenant_id`, indexée, identifie à qui appartient chaque enregistrement. Toutes les requêtes SQL doivent inclure un filtre `WHERE tenant_id = :id` pour garantir l'isolation logique.

Les avantages sont nombreux : déploiement simplifié (une seule base de données à maintenir), migrations de schéma appliquées une seule fois, consommation de ressources optimisée. Les risques résident dans les bugs de filtrage (oubli du filtre `tenant_id` dans une requête) et dans la moindre isolation en cas d'attaque ou d'erreur. BCaaS adopte cette stratégie pour son MVP, en s'appuyant sur des mécanismes de sécurité au niveau applicatif (middleware d'injection automatique du `tenant_id`) pour réduire les risques d'oubli.

b) Schéma PostgreSQL séparé par tenant

Chaque tenant dispose de son propre schéma PostgreSQL (un *namespace* de tables) au sein d'une même base de données. L'isolation est physique au niveau des données mais les tenants partagent toujours le moteur de base de données et ses ressources.

Cette approche offre une meilleure isolation (il est impossible d'accéder aux données d'un autre schéma sans changer explicitement de `search_path`), des sauvegardes par tenant plus aisées, et une capacité à personnaliser le schéma par tenant. En contrepartie, les migrations doivent être appliquées à chaque schéma séparément, ce qui complexifie les déploiements.

c) Base de données séparée par tenant

Chaque tenant dispose de sa propre instance de base de données. C'est le niveau d'isolation le plus fort : une défaillance ou une compromission de la base d'un tenant n'affecte pas les autres. Cette stratégie est requise dans les secteurs très régulés (banque, santé) où les données doivent être physiquement séparées.

Le coût opérationnel est significatif : N tenants impliquent N bases à provisionner, surveiller, sauvegarder et mettre à jour. Elle est donc rarement justifiée pour un grand nombre de petits tenants.

2.2.3 Analogie avec les réseaux virtuels (VLAN)

L'analogie entre le multi-tenant et les VLAN (*Virtual Local Area Networks*) est particulièrement éclairante. Un VLAN permet de segmenter logiquement un réseau physique unique en plusieurs réseaux virtuels isolés. Des équipements physiquement connectés au même commutateur peuvent appartenir à des VLAN différents et ne pas pouvoir communiquer entre eux, comme s'ils étaient sur des réseaux entièrement séparés.

De même, dans une architecture multi-tenant à schéma partagé, les données de différents tenants coexistent sur la même infrastructure physique (même serveur, même base de données) mais sont isolées logiquement par leur `tenant_id`, exactement comme les trames Ethernet sont isolées par leur `tag` VLAN (norme IEEE 802.1Q). BCaaS exploite cette analogie pour conceptualiser et documenter son architecture.

2.3 Le paradigme *as-a-Service* et le *Business as a Service*

2.3.1 L'évolution des modèles *as-a-Service*

Le cloud computing a popularisé une famille de modèles de fourniture de services informatiques regroupés sous le terme générique *as-a-Service*. L'idée centrale est de fournir une capacité informatique sous forme de service accessible via une interface standardisée, déchargeant le consommateur de la responsabilité de construire et d'opérer l'infrastructure sous-jacente. Ces

Table 2.3: Comparaison des modèles *as-a-Service* et positionnement de BCaaS

Modèle	Ce qui est fourni	Exemples concrets	Analogie BCaaS
IaaS	Infrastructure (serveurs, stockage, réseau)	AWS EC2, Azure VMs, GCP Compute	—
PaaS	Plateforme d'exécution + middleware	Heroku, Vercel, Render	—
SaaS	Application complète prête à l'emploi	Salesforce, Gmail, Notion	—
BaaS (BCaaS)	Briques métier réutilisables via API	BCaaS acteurs, règles, facturation	Couche métier partageable

trois modèles forment une progression : IaaS délègue la gestion du matériel, PaaS y ajoute

celle du système d'exploitation et des middlewares, SaaS va jusqu'à déléguer la gestion de l'application elle-même. Le consommateur n'a plus qu'à utiliser le service, sans se préoccuper de ce qui se passe en dessous.

2.3.2 Le *Business as a Service* (BaaS)

Le *Business as a Service* (BaaS) transpose cette logique au niveau des fonctions métier. Plutôt que de déléguer de l'infrastructure ou une application, il s'agit d'exposer comme services réutilisables les briques fonctionnelles communes à tout système d'information professionnel.

Ces briques récurrentes comprennent typiquement :

- La gestion des acteurs : utilisateurs, clients, fournisseurs, partenaires et leurs rôles respectifs.
- La gestion des ressources : produits, services, inventaires, tarifs.
- La facturation et la comptabilité : devis, commandes, factures, paiements, rapprochements.
- Les règles métier : politiques de prix, conditions d'accès, workflows d'approbation.
- La traçabilité et l'audit : journaux d'événements, historiques, conformité réglementaire.

Au lieu que chaque projet réimplémente ces fonctions depuis zéro, le BaaS propose de les centraliser dans un socle commun consommable via des API. BCaaS est une mise en œuvre concrète de ce paradigme, conçue comme socle commun à plusieurs instances métier distinctes pouvant appartenir à des secteurs différents.

2.3.3 Avantages et défis du BaaS

Avantages. Réduction drastique du temps de développement des projets consommateurs, homogénéité des comportements métier entre projets, centralisation de la maintenance et des corrections de bogues, et mutualisation des coûts d'infrastructure.

Défis. Le socle doit être suffisamment générique pour couvrir des cas d'usage variés sans être si abstrait qu'il devient inutilisable. La gestion des personnalisations par tenant (règles métier spécifiques à un client) et la compatibilité ascendante de l'API sont des points de vigilance critiques.

2.4 Architecture hexagonale et *Domain-Driven Design*

2.4.1 L'architecture hexagonale (*Ports & Adaptateurs*)

Proposée par Alistair Cockburn en 2005 [9], l'architecture hexagonale également appelée *Ports and Adapters* repose sur un principe fondamental : le cœur métier d'une application ne doit dépendre d'aucun détail technique. La base de données, le framework web, la messagerie asynchrone, les systèmes externes sont tous des détails susceptibles de changer, et le domaine métier ne doit pas en être affecté.

L'architecture se structure autour de trois zones :

Le domaine (hexagone central) Contient la logique métier pure : les entités, les règles, les calculs. Il est exprimé en langage métier et ne dépend d'aucune technologie.

Les ports Interfaces (au sens de contrats ou d'abstractions) définies par le domaine pour interagir avec l'extérieur. Un port entrant (*driving port*) représente ce que le domaine offre aux utilisateurs (commandes, requêtes). Un port sortant (*driven port*) représente ce dont le domaine a besoin de l'infrastructure (persistance, notifications).

Les adaptateurs Implémentations concrètes des ports. Un adaptateur REST traduit les requêtes HTTP en appels de commandes métier. Un adaptateur PostgreSQL traduit les appels de persistance en requêtes SQL. Les adaptateurs sont interchangeables sans toucher au domaine.

Les bénéfices de cette architecture sont considérables pour un projet comme BCaaS. Le cœur métier peut être testé indépendamment de toute infrastructure (tests unitaires rapides, sans base de données ni serveur HTTP). On peut changer de base de données, de framework web ou de système de messagerie sans réécrire la logique métier. Les adaptateurs peuvent être développés et remplacés indépendamment.

2.4.2 Le *Domain-Driven Design* (DDD)

Le *Domain-Driven Design*, formalisé par Eric Evans dans son ouvrage fondateur *Domain-Driven Design: Tackling Complexity in the Heart of Software* (2003) [?], fournit un ensemble de *patterns* et de pratiques pour modéliser des domaines métier complexes. Il s'articule autour de plusieurs concepts clés :

Le langage omniprésent (*Ubiquitous Language*) Développeurs et experts métier utilisent exactement le même vocabulaire pour désigner les concepts du domaine. Ce langage partagé se retrouve dans le code, les tests, la documentation et les discussions. Il élimine les ambiguïtés et les traductions coûteuses entre monde technique et monde métier.

Les entités (*Entities*) Objets métier possédant une identité unique et persistante à travers le temps, indépendamment de leurs attributs. Un client reste le même client même si son adresse change : c'est l'identité qui compte, pas les valeurs.

Les objets-valeur (*Value Objects*) Objets définis uniquement par leurs attributs, sans identité propre. Une adresse postale, un montant en devise, une plage de dates sont des objets-valeur : deux instances identiques sont interchangeables.

Les agrégats (*Aggregates*) Grappes d'entités et d'objets-valeur traitées comme une unité cohérente pour les modifications. Chaque agrégat a une racine (*Aggregate Root*) qui est le seul point d'accès depuis l'extérieur, garantissant la cohérence des invariants métier.

Les services de domaine (*Domain Services*) Opérations métier qui n'appartiennent naturellement à aucune entité ou objet-valeur, telles qu'un calcul de TVA impliquant des règles fiscales complexes.

Les contextes délimités (*Bounded Contexts*) Frontières explicites dans lesquelles un modèle de domaine particulier s'applique. Un même mot peut avoir des significations différentes

dans des contextes différents : un « client » dans le contexte commercial n'est pas le même concept que dans le contexte du support technique.

2.4.3 DDD et architecture hexagonale : une synergie naturelle

L'architecture hexagonale et le DDD se complètent naturellement. Le DDD fournit le modèle conceptuel du domaine (ce qu'on modélise et pourquoi), tandis que l'architecture hexagonale fournit la structure technique pour implémenter ce modèle de manière isolée et testable (comment on l'organise dans le code).

Ensemble, ils permettent de construire un **monolithe modulaire** : une application unique et cohérente, mais découpée en modules aux frontières explicites (les *Bounded Contexts*) et aux interfaces bien définies (les ports). Cette structure prépare une éventuelle migration vers les microservices : chaque module peut, à terme, être extrait et déployé indépendamment sans réécriture du domaine, car les dépendances inter-modules passent déjà par des interfaces, non par des appels directs.

2.5 Positionnement de BCaaS

2.5.1 Ce que la littérature fournit déjà

La revue de la littérature met en évidence que chacun des éléments conceptuels mobilisés par BCaaS existe et est documenté indépendamment :

- Les modèles en couches OSI et TCP/IP sont des standards établis depuis plusieurs décennies, avec une littérature abondante et des implémentations universelles.
- Les stratégies multi-tenant sont documentées dans la littérature académique et dans les guides de bonnes pratiques des fournisseurs cloud (Microsoft Azure, AWS).
- Le paradigme *as-a-Service* est largement décrit et analysé, notamment dans les travaux du NIST sur le cloud computing [8].
- L'architecture hexagonale et le DDD sont des méthodologies reconnues, avec des communautés actives et des implémentations de référence dans de nombreux langages.

2.5.2 Le *gap* identifié

Cependant, la littérature existante traite ces éléments séparément, dans des silos disciplinaires distincts : les modèles réseau dans la littérature des systèmes distribués, le multi-tenant dans la littérature SaaS, l'architecture hexagonale dans la littérature du génie logiciel. Il n'existe pas, à notre connaissance, de travail proposant une grille de lecture unifiée qui utilise les métaphores réseau pour conceptualiser et documenter un système logiciel multi-tenant à architecture hexagonale.

2.5.3 La contribution de BCaaS

L'originalité de BCaaS ne réside donc pas dans l'invention de l'un de ces éléments, mais dans leur **unification sous une grille de lecture réseau cohérente et systématique**. Cette grille se manifeste à travers plusieurs analogies structurantes :

La pile métier comme pile protocolaire Les couches de l'application (présentation, application, domaine, infrastructure) sont interprétées comme les couches d'une pile réseau, avec des responsabilités distinctes et des interfaces explicites entre couches.

L'isolation multi-tenant comme VLAN Le partage d'infrastructure avec étiquetage logique des flux par `tenant_id` est l'exact équivalent logiciel du marquage de trames par *tag* VLAN dans les réseaux commutés (IEEE 802.1Q).

La facture comme accusé de réception Le cycle de vie d'une facture (émission, accusé, paiement, rapprochement) est modélisé sur le *pattern* ACK/NACK des protocoles réseau fiables : chaque étape confirme la bonne réception et le traitement du message précédent.

L'administration comme plan de contrôle SDN La séparation entre le plan de données (traitement des transactions métier) et le plan de contrôle (configuration des règles, gestion des tenants, supervision) reflète l'architecture *Software-Defined Networking*, où le plan de contrôle est découplé du plan de données pour plus de flexibilité.

Cette cohérence conceptuelle (utiliser un seul cadre métaphorique pour relier des concepts habituellement traités séparément) constitue l'apport central du présent travail. Elle offre un langage commun accessible aussi bien aux profils techniques (développeurs, architectes) qu'aux profils familiers des réseaux, et facilite la communication entre équipes aux expertises différentes.

La transposition de ces métaphores en décisions d'architecture concrètes sera détaillée au chapitre 6, qui présente le modèle conceptuel de BCaaS.

Synthèse

Ce chapitre a passé en revue les quatre piliers conceptuels sur lesquels repose BCaaS : les modèles en couches OSI/TCP-IP, les architectures multi-tenant, le paradigme *as-a-Service* étendu au niveau métier, et l'architecture hexagonale couplée au *Domain-Driven Design*. Pour chacun de ces piliers, nous avons présenté les concepts fondamentaux, leurs avantages et leurs limites, ainsi que leurs applications concrètes dans le contexte de BCaaS.

La section 2.5 a montré que l'originalité de BCaaS tient à la combinaison et à l'unification de ces approches sous une grille de lecture réseau cohérente une perspective que la littérature n'exploite pas jusqu'ici de manière systématique. Le chapitre suivant s'appuiera sur cet état de l'art pour présenter le modèle d'architecture détaillé de BCaaS et expliciter comment chaque analogie réseau se traduit en décisions de conception concrètes.

CHAPITRE 3

Analogie formelle entre réseaux et métier

Ce chapitre constitue le cœur conceptuel de notre projet. Loin d'être une simple métaphore, l'approche développée ici établit un isomorphisme formel entre l'ingénierie des protocoles réseaux et la structuration des systèmes d'information métiers (Business Core).

En transposant les concepts de la pile OSI, de la segmentation logicielle (VLAN), de l'encapsulation et du contrôle de flux, nous posons les fondements d'une infrastructure métier industrialisée et résiliente. Cette grille de lecture technique dictera l'ensemble de nos choix architecturaux et stratégies de déploiement.

3.1 Table de correspondance réseau–métier

Le tableau 3.1 formalise la mise en correspondance. Chaque concept réseau y trouve un équivalent métier précis dans BCaaS, ce qui confère à l'architecture sa cohérence d'ensemble.

Table 3.1: Correspondance formelle entre concepts réseaux et concepts métier

Concept réseau	Définition réseau	Équivalent métier dans BCaaS
Adresse IP / MAC	Identifiant unique d'un hôte ou d'une interface	URI de l'API Métier + Identifiant de la ressource (resource_id)
En-tête de paquet	Métadonnées d'acheminement et de contrôle	Contexte de requête (tenant_id , actor_id , trace_id)
Payload / SDU	Données applicatives transportées	Corps de la requête métier (Payload JSON)
Encapsulation	Ajout d'en-têtes à chaque couche descendante	Enrichissement progressif du contexte métier par les filtres et intercepteurs
Routage	Sélection du chemin vers la destination	Résolution dynamique du flux (Tenant → Service → Handler exécutant)
NAT / Proxy	Traduction d'adresses et masquage d'architecture	API Gateway : centralisation, réécriture et normalisation des requêtes
VLAN (802.1Q)	Isolation logique sur un domaine partagé	Isolation stricte par tenant_id sur un schéma de base de données unique
TCP (Fiabilité)	Transport orienté connexion avec acquittement	Échanges REST synchrones avec mécanismes de retry, timeout et idempotence
QoS	Priorisation du trafic et garanties de ressources	Politiques de SLA par tenant et niveaux de priorité des transactions métiers
TTL	Durée de vie maximale d'un paquet sur le réseau	Durée de validité du jeton JWT / Expiration du contexte de transaction
Control plane	Gestion des routes et politiques (BGP, OSPF)	Module Admin : configuration des tenants, des work-flows et des règles SLA
Data plane	Acheminement effectif des paquets de données	Moteur d'exécution : traitement des transactions et persistance des données
Firewall	Filtrage de paquets selon des règles de sécurité	Spring Security : validation des jetons JWT et contrôle d'accès RBAC
Sonde / Wireshark	Capture et analyse du trafic en temps réel	Système de traçabilité distribuée et d'observabilité des flux métiers

3.2 Émetteur, récepteur, protocole et message

Pour formaliser la dynamique des flux au sein de la plateforme BCaaS, il ne suffit pas de cartographier les composants statiques ; il est indispensable de modéliser les interactions selon la théorie classique des systèmes de communication. Tout échange métier au sein du *Business Core* est ainsi traité comme une transmission de données sur un réseau, structurée autour du modèle fondamental : émetteur – récepteur – protocole – message – accusé – canal.

Chaque entité de notre modèle métier s'intègre rigoureusement dans cette chaîne de communication :

- **Émetteur (source de la requête)** : Le *Consommateur de Service* (CdS). Il s'agit d'un acteur (**Actor**), client ou composant du système, qui initie l'interaction en émettant une demande de service formelle (**ServiceRequest**).

- **Récepteur (destination de la requête, source de la réponse)** : Le *Fournisseur de Service* (FdS). C'est l'acteur (**Actor**) ou le sous-système applicatif qui accepte la charge utile, prend en charge l'exécution de la logique métier et finalise la prestation attendue.
- **Protocole (règles du métier)** : Les **BusinessRule**. Représentées sous forme de paires clé-valeur dynamiques, elles définissent la syntaxe, la sémantique et les contraintes opérationnelles que le message doit respecter pour être validé et traité par le récepteur, jouant le rôle d'un protocole de communication réseau.
- **Message / requête** : La demande de service (**ServiceRequest**). C'est l'unité d'information (le paquet) qui circule. Elle transporte les données fonctionnelles (*Payload*), et encapsule également un **traceId** (identifiant unique du cycle de vie de la requête) et un **correlationId** (identifiant de liaison inter-services) permettant d'assurer la traçabilité complète et l'auditabilité du flux en temps réel.
- **Accusé de réception (ACK)** : La facture métier (**Invoice**). Générée de manière immuable et automatique dès la finalisation du traitement par le Fournisseur de Service, elle confirme la bonne exécution de l'opération et notifie l'émetteur du succès de la transaction.
- **Canal** : L'API REST sécurisée de bout en bout par des jetons applicatifs JWT. Le canal fournit le support logique nécessaire au transport des messages, garantissant la confidentialité et l'intégrité de l'échange.

3.3 Flux de traitement d'une requête

Le traitement d'une requête suit le modèle OSI descendant (émission) puis ascendant (réception) : le payload métier est progressivement enrichi de son en-tête tenant, de son contexte d'exécution, puis de ses permissions et de son niveau de SLA, avant d'atteindre le service métier qui produit la réponse et publie les événements associés (notification, audit). La figure 3.1 illustre ce flux bout-en-bout.

3.4 La pile protocolaire métier en cinq couches

De l'analogie découle une décomposition de BCaaS en cinq couches protocolaires, en correspondance directe avec le modèle OSI (figure 3.2). Chaque couche n'expose qu'un service bien défini à la couche supérieure.

3.4.1 Couche 1 — Infrastructure

Substrat technique : base de données PostgreSQL avec migrations Liquibase, cache Redis et bus de messages. Elle correspond aux couches physique et liaison : elle transporte les données sans en connaître la signification.

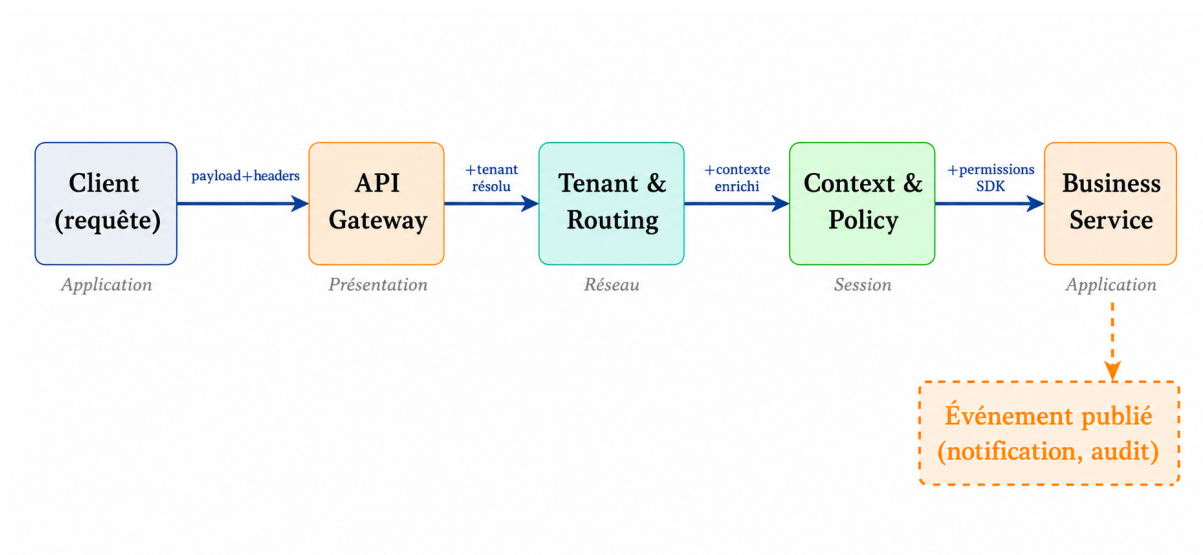


Figure 3.1: Flux bout-en-bout d’une requête dans BCaaS : Client → API Gateway → Tenant & Routing → Context & Policy → Business Service, avec correspondance OSI descendante/as-cendante et publication d’événements (notification, audit).

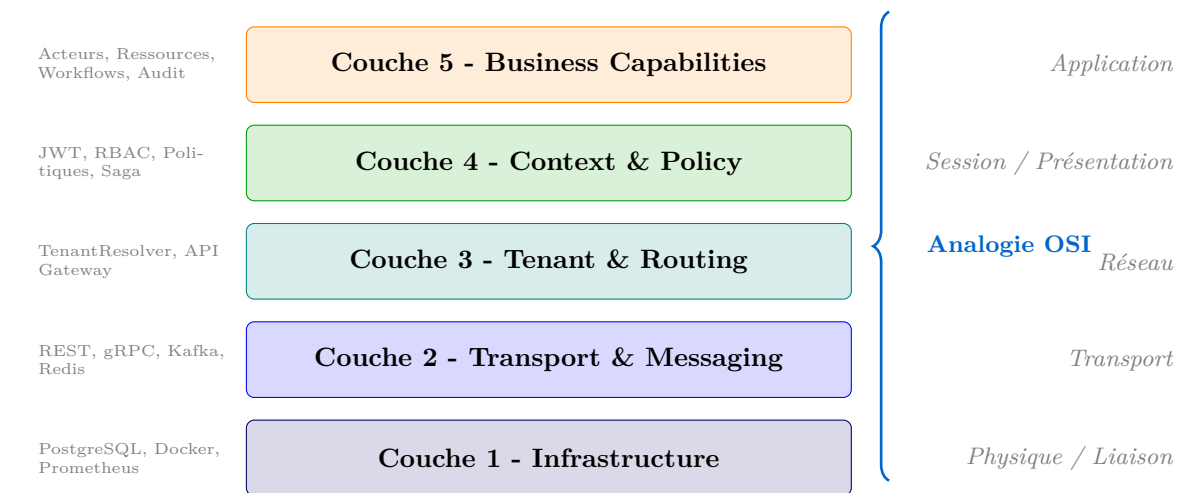


Figure 3.2: Pile protocolaire métier de BCaaS et correspondance avec le modèle OSI

3.4.2 Couche 2 — Transport & Messaging

Analogue à la couche transport (TCP/UDP). Elle gère le mode d’échange (synchrone via REST, ou asynchrone via le bus d’événements), les stratégies de retry, les timeouts et l’idempotence. Le choix *REST* $\equiv$ *TCP* (fiable, ordonné) ou *événement* $\equiv$ *UDP* (rapide, au mieux) est explicite dans la conception.

3.4.3 Couche 3 — Tenant & Routing

Analogue à la couche réseau. Le *Tenant Resolver* joue le rôle d’un routeur : à partir de l’en-tête de la requête (le claim `tenantId` du JWT), il détermine le contexte d’exécution et oriente la requête vers le bon service. C’est ici que réside la logique de découverte et d’isolation.

3.4.4 Couche 4 — Context & Policy

Analogue aux couches session et présentation. Cette couche valide le token JWT, décode les rôles et permissions, applique les politiques de sécurité propres au tenant et détermine le niveau de service. C'est ici qu'est construit l'« en-tête métier » complet de la requête.

3.4.5 Couche 5 — Business Capabilities

Couche applicative. Elle contient les services métier génériques (acteurs, métiers, services, ressources, règles, factures, audit), qui opèrent sur des données configurables par tenant sans connaissance des couches inférieures.

3.5 Le multi-tenant comme réseau virtuel (VLAN)

Analogie : VLAN ↔ Tenant

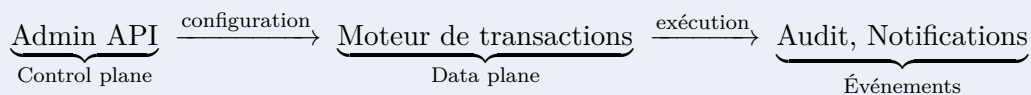
Dans un réseau, un VLAN isole logiquement plusieurs groupes de machines sur la même infrastructure physique ; chaque trame porte un *tag* (802.1Q) identifiant son appartenance. Dans BCaaS, chaque requête porte un `tenant_id` qui isole les données, les règles et les workflows d'un client sur l'infrastructure partagée.

$\text{tag VLAN} \equiv \text{tenant_id}, \quad \text{réseau physique} \equiv \text{infrastructure partagée}.$

3.6 Plan de contrôle et plan de données

Analogie : SDN ↔ Administration BCaaS

Dans les réseaux définis par logiciel (SDN), le *control plane* configure de façon centralisée les règles de routage, tandis que le *data plane* achemine les paquets. Dans BCaaS, l'interface d'administration (control plane) configure tenants, règles métier, catalogues et niveaux de service ; le moteur d'exécution (data plane) traite les transactions en appliquant ces configurations, et émet les événements d'audit et de notification.



3.7 Routage applicatif et recherche de chemin logique

Analogie : Table de Routage ↔ Workflow Engine / Handler Resolver

Dans les infrastructures réseaux, le routage consiste à déterminer le chemin optimal qu'un paquet doit emprunter à travers des nœuds intermédiaires pour atteindre sa destination, en se basant sur des tables de routage dynamiques (alimentées par OSPF ou BGP).

Dans la plateforme BCaaS, nous transposons ce mécanisme sous la forme d'un *Handler Resolver* couplé à un moteur de règles. Lorsqu'une **ServiceRequest** franchit les couches de sécurité, le cœur du système inspecte les métadonnées de la requête (le type de service et l'action demandée). À l'instar d'un routeur effectuant un *longest prefix match* sur une adresse IP, le moteur BCaaS effectue une résolution dynamique pour aiguiller le message vers l'instance de composant métier (*Service Handler*) configurée pour ce tenant spécifique.

3.8 Contrôle de flux et qualité de service (QoS)

Analogie : QoS / Token Bucket ↔ Rate Limiting et SLA

La mise en commun des ressources physiques au sein d'une infrastructure partagée expose le système au syndrome du *Noisy Neighbor* (un utilisateur saturant les ressources au détriment des autres). En ingénierie des réseaux, ce problème est résolu par des politiques de Qualité de Service (QoS) et des algorithmes de lissage de trafic comme le *Token Bucket*, qui limitent le débit d'un flux à une bande passante allouée.

Au niveau de BCaaS, cette gestion des congestions est implémentée par un mécanisme de *Rate Limiting* adossé aux contrats de niveau de service (*SLA*) des tenants. Chaque requête entrante est soumise à un compteur glissant (géré en cache haute performance via Redis). Si un tenant dépasse le quota de transactions par seconde (TPS) stipulé par ses règles de configuration, le système applique un contrôle de flux strict en rejetant les requêtes excédentaires avec un code statut HTTP 429 (*Too Many Requests*), préservant ainsi la disponibilité globale du *Business Core*.

CHAPITRE 4

Analyse et spécification des besoins

4.1 Besoins fonctionnels

Les besoins fonctionnels sont identifiés par un code F-XXX-NNN. Les tableaux 4.1 à 4.3 les regroupent par domaine.

Table 4.1: *Besoins fonctionnels — Identités (Tenant, User, Actor)*

ID	Fonctionnalité	Contraintes
F-TEN-001	Provisionner un tenant (inscription publique)	Nom et domaine obligatoires, <code>tenant_id</code> UUID généré
F-USR-001	Inscrire un utilisateur (register)	Email unique, mot de passe chiffré (BCrypt)
F-USR-002	Authentifier (login)	Retourne un JWT portant rôles et <code>tenantId</code>
F-USR-003	Gérer les rôles d'un utilisateur	Réservé au rôle ADMIN
F-ACT-001	Créer un acteur lié à un utilisateur	Un utilisateur peut avoir plusieurs acteurs
F-ACT-002	Filtrer les acteurs par rôle	—

Table 4.2: *Besoins fonctionnels — Métiers (Business, Rule, Catalogue, Resource, Media, Portfolio)*

ID	Fonctionnalité	Contraintes
F-BUS-001	Créer / modifier un métier	Nom et domaine obligatoires ; rattaché au tenant
F-BUS-002	Rechercher par domaine	—
F-BRU-001	Définir une règle métier (clé-valeur)	Rattachée à un Business
F-SCA-001	Créer un service au catalogue	Prix de base + devise validée
F-SCA-002	Filtrer les services disponibles	<code>isAvailable</code>
F-RES-001	Gérer une ressource (stock)	Quantité ≥ 0
F-MED-001	Associer un média à un métier	Type + URL
F-POR-001	Gérer un portfolio de compétences	1 portfolio par acteur ; relation N:M avec Business

Table 4.3: *Besoins fonctionnels — Transactions (ServiceRequest, Invoice, Message, Notification)*

ID	Fonctionnalité	Contraintes
F-SRE-001	Créer une demande de service (CdS)	Consumer, provider et service du catalogue requis
F-SRE-002	Faire évoluer le statut	FSM : accept / start / fulfill / cancel
F-SRE-003	Suivre par statut / acteur	—
F-INV-001	Générer une facture	Automatique à la finalisation (ACK)
F-INV-002	Payer / annuler une facture	FSM : pay / cancel
F-MSG-001	Échanger des messages sur une demande	Temps réel (publication/abonnement)
F-NOT-001	Notifier un utilisateur	Compteur de non-lus, marquage lu
F-AUD-001	Tracer les événements (audit)	Par <code>traceId</code> , acteur, tenant
F-ANA-001	Calculer les KPIs d'un tenant	—

4.2 Besoins non fonctionnels

Le sujet impose un ensemble exigeant de contraintes non fonctionnelles. Le tableau 4.4 en précise le statut dans le présent MVP, dans une logique de transparence : ce qui n'est pas pleinement réalisé est repris comme perspective au chapitre 13.

Table 4.4: *Besoins non fonctionnels et statut dans le MVP*

Besoin	Description	Statut MVP
Multi-tenant	Isolation des données par tenant sur infrastructure partagée	Réalisé
Sécurité	Authentification JWT, mots de passe chiffrés, RBAC	Réalisé
Multi-devises	Support XAF, XOF, NGN, KES, GHS, USD, EUR, GBP	Réalisé
Documentation API	Spécification OpenAPI / Swagger UI	Réalisé
Conteneurisation	Docker / docker-compose	Réalisé
Modularité (DDD / hexagonal)	Monolithe modulaire, microservices-ready	Réalisé
Temps réel	Messagerie instantanée sur les demandes	Partiel (Redis pub/sub)
Événementiel	Bus d'événements (notifications, audit)	Partiel (infra posée)
Géo-référencement	PostGIS, géolocalisation native	Perspective
Offline-first / PWA	Application installable et hors-ligne	Perspective
Internationalisation	i18n web et mobile	Perspective
Réactif	WebFlux + R2DBC non bloquant	Perspective
2FA	Double authentification	Perspective
Mobile natif	React Native	Perspective

4.3 Diagramme de contexte



Figure 4.1: *Diagramme de contexte du système*

CHAPITRE 5

Modélisation mathématique

Ce chapitre propose une formalisation ensembliste du système, de ses contraintes et de ses indicateurs. Cette modélisation explicite les invariants que l'implémentation doit garantir et fournit un langage rigoureux pour raisonner sur le matching et les performances.

5.1 Ensembles fondamentaux

Le système est décrit par les ensembles d'entités suivants, tous identifiés par des UUID :

T (Tenants), U (Users), A (Actors), B (Businesses), S (ServiceCatalogues),
 R (Resources), Po (Portfolios), SR (ServiceRequests), I (Invoices), BR (BusinessRules).

L'ensemble des devises supportées est fini :

$$C = \{XAF, XOF, NGN, KES, GHS, USD, EUR, GBP\}.$$

5.1.1 Décomposition des acteurs

$$A = A_c \cup A_p, \quad A_c \cap A_p \neq \emptyset,$$

où A_c (consommateurs) et A_p (fournisseurs) peuvent se recouper : un même acteur peut à la fois consommer et fournir des services.

5.1.2 Attributs des entités principales

$$t = (id_T, name, domain, isActive, createdAt) \in T,$$

$$u = (id_U, tenant, email, roles, isActive, \dots) \in U,$$

$$a = (id_A, user, role, bio, isActive) \in A,$$

$$sr = (id_{SR}, tenant, consumer, provider, catalogue, status, traceId, \dots) \in SR,$$

$$i = (id_I, serviceRequest, amount, currency, status, issuedAt, paidAt) \in I.$$

5.2 Variables de décision et fonction objectif

5.2.1 Variables

Variable binaire de matching :

$$x_{a_c, a_p, b} \in \{0, 1\}, \quad \forall (a_c, a_p, b) \in A_c \times A_p \times B,$$

valant 1 si le consommateur a_c est mis en relation avec le fournisseur a_p pour le métier b .

Variable d'état d'une demande :

$$\sigma_{sr} \in \Sigma_{SR} = \{\text{PENDING, ACCEPTED, IN_PROGRESS, FULFILLED, CANCELLED}\}.$$

5.3 Contraintes du système

5.3.1 Contraintes de cardinalité

$$\forall u_1, u_2 \in U, u_1 \neq u_2 : u_1.\text{email} \neq u_2.\text{email} \quad (\text{unicité de l'email}) \quad (5.1)$$

$$\forall a \in A, \exists! po \in Po : po.\text{actor} = a \quad (\text{Actor} \rightarrow \text{Portfolio}, 1:1) \quad (5.2)$$

$$\forall po \in Po : po.\text{businesses} \subseteq B \quad (\text{Portfolio} \rightarrow \text{Business}, N:M) \quad (5.3)$$

$$\forall sr \in SR, \exists! i \in I : i.\text{serviceRequest} = sr \quad (\text{ServiceRequest} \rightarrow \text{Invoice}, 1:1) \quad (5.4)$$

5.3.2 Contraintes métier

Validité d'une demande :

$$\text{valid}(sr) \iff \begin{cases} sr.\text{consumer.role} = \text{CONSUMER}, \\ sr.\text{provider.role} = \text{PROVIDER}, \\ sr.\text{provider.isActive} = \text{vrai}. \end{cases}$$

Compatibilité de matching :

$$\text{compatible}(a_c, a_p, b) \iff \exists po \in Po : (po.\text{actor} = a_p \wedge b \in po.\text{businesses}).$$

Contrainte de devise :

$$\forall i \in I : i.\text{currency} \in C, \quad \forall s \in S : s.\text{currency} \in C.$$

Contrainte d'isolation multi-tenant (invariant de sécurité) :

$$\forall sr \in SR : sr.\text{tenant} = \tau \implies (sr.\text{consumer.user.tenant} = \tau \wedge sr.\text{catalogue.business.tenant} = \tau).$$

5.3.3 Contrainte de workflow : machine à états

Soit $\delta_{SR} : \Sigma_{SR} \times \mathcal{A} \rightarrow \Sigma_{SR}$ la fonction de transition, avec $\mathcal{A} = \{\text{create, accept, start, fulfill, cancel}\}$. Les transitions valides sont données au tableau 5.1.

Table 5.1: *Transitions valides de la machine à états ServiceRequest*

État courant	Action	État suivant
$\emptyset$	create	PENDING
PENDING	accept	ACCEPTED
PENDING	cancel	CANCELLED
ACCEPTED	start	IN_PROGRESS
ACCEPTED	cancel	CANCELLED
IN_PROGRESS	fulfill	FULFILLED
IN_PROGRESS	cancel	CANCELLED

5.4 Algorithmes et complexité

5.4.1 Recherche des fournisseurs d'un métier

```

1 Entree : b (business cible)
2 Sortie : ensemble des providers compatibles
3 1: Resultat <- vide
4 2: pour chaque po dans Po :
5 3:     si b appartient a po.businesses :
6 4:         a <- po.actor
7 5:         si a.role = PROVIDER et a.isActive : Resultat <- Resultat U {a}
8 6: retourner Resultat

```

Listing 5.1: *FindProvidersByBusiness*

Complexité : $O(|Po|)$ dans le pire cas, $O(1)$ amorti avec un index sur les métiers du portfolio.

5.4.2 Recherche par filtres combinés

Pour un ensemble de filtres $F = \{f_1, \dots, f_n\}$, le résultat est l'intersection

$$\text{result}(F) = \bigcap_{f \in F} \text{applyFilter}(f, \text{Univers}),$$

optimisée en appliquant d'abord le filtre le plus sélectif :

$$\text{selectivity}(f) = \frac{|\text{applyFilter}(f, \text{Univers})|}{|\text{Univers}|}, \quad \text{ordre croissant de selectivity.}$$

5.5 Indicateurs et métriques (KPIs)

Sur une période $[t_1, t_2]$, on définit :

$$T_{\text{conv}} = \frac{N_{\text{fulfilled}}}{N_{\text{requests}}} \times 100\%, \quad PA = \frac{CA}{N_{\text{paid}}},$$

où le chiffre d'affaires est

$$CA(t_1, t_2) = \sum_{\substack{i \in I : \sigma_i = \text{PAID} \\ i.\text{paidAt} \in [t_1, t_2]}} i.\text{amount},$$

et le temps moyen de réalisation

$$T_{\text{compl}} = \frac{1}{N_{\text{fulfilled}}} \sum_{sr \in SR : \sigma_{sr} = \text{FULFILLED}} (sr.\text{fulfilledAt} - sr.\text{requestedAt}).$$

La part d'un métier b dans le chiffre d'affaires s'écrit $\text{Share}_b = CA_b / CA \times 100\%$, et la croissance mensuelle $\text{Growth}_{\text{MoM}}(t) = (CA(t) - CA(t-1)) / CA(t-1) \times 100\%$. Ces indicateurs sont calculés par le service d'analytique

5.6 Architecture en couches : composition de fonctions

Ce modèle formalise le traitement d'une requête à travers les cinq couches de BCaaS. Chaque couche est représentée par une fonction de transformation, et le traitement complet est leur composée.

5.6.1 Définition formelle

On associe à chaque couche $i \in \{1, \dots, 5\}$ une fonction :

$f_1 : r \mapsto r_1$	Infrastructure (connexion BD, cache)
$f_2 : r_1 \mapsto r_2$	Transport & Messaging (REST ou Kafka)
$f_3 : r_2 \mapsto r_3$	Tenant & Routing (résolution <code>tenant_id</code>)
$f_4 : r_3 \mapsto r_4$	Context & Policy (JWT, RBAC, SLA)
$f_5 : r_4 \mapsto r_5$	Business Capabilities (logique métier)

Le traitement complet d'une requête r est la **composée** de ces cinq fonctions :

$$\text{BCaaS}(r) = (f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1)(r) = f_5(f_4(f_3(f_2(f_1(r)))))$$

5.6.2 Propriété : encapsulation stricte

Pour tout $i \in \{1, \dots, 5\}$, la fonction f_i ne reçoit en entrée que r_{i-1} et est indépendante de f_j pour tout $j > i$:

$$f_i : \mathcal{R}_{i-1} \longrightarrow \mathcal{R}_i, \quad f_i \perp f_j \text{ pour } j > i.$$

Modifier f_5 n'affecte jamais f_1 : c'est la traduction formelle du principe d'encapsulation du modèle OSI.

5.6.3 Exemple

Considérons une **ServiceRequest** de création de métier (`POST /api/businesses`) émise par un acteur authentifié. La requête r traverse les cinq couches : f_1 ouvre la connexion PostgreSQL, f_2 sélectionne le transport REST synchrone, f_3 résout le `tenant_id` depuis l'en-tête `X-Tenant-ID`, f_4 valide le token JWT et le rôle `ADMIN`, et f_5 persiste l'entité et retourne `201 Created` :

$$\text{BCaaS}(r) = f_5(f_4(f_3(f_2(f_1(r)))))) = r_5 = 201 \text{ Created}.$$

Si f_4 échoue (JWT expiré), la composition s'arrête en r_4 et retourne `401 Unauthorized`, sans jamais atteindre f_5 .

5.7 Isolation multi-tenant : théorie des ensembles

Ce modèle formalise l'étanchéité des données entre les tenants de l'ensemble T (défini à la section 5.1). Il constitue la preuve ensembliste globale de la contrainte d'isolation posée à la section 5.3.

5.7.1 Définitions

On note $\mathcal{E}$ l'ensemble de toutes les lignes présentes en base de données (tables U , A , B , SR , I , etc. confondues). On définit la fonction d'appartenance $\pi : \mathcal{E} \rightarrow T$ par :

$$\pi(e) = \tau_k \iff \text{la colonne } \texttt{tenant_id} \text{ de } e \text{ vaut } \tau_k.$$

L'ensemble des données du tenant τ_k est :

$$\mathcal{D}(\tau_k) = \{ e \in \mathcal{E} \mid \pi(e) = \tau_k \}.$$

5.7.2 Propriété : isolation totale

$$\forall \tau_i, \tau_j \in T, \quad \tau_i \neq \tau_j \implies \mathcal{D}(\tau_i) \cap \mathcal{D}(\tau_j) = \emptyset.$$

Les ensembles $\mathcal{D}(\tau_1), \dots, \mathcal{D}(\tau_n)$ forment une **partition** de $\mathcal{E}$:

$$\mathcal{E} = \biguplus_{k=1}^n \mathcal{D}(\tau_k) = \mathcal{D}(\tau_1) \uplus \mathcal{D}(\tau_2) \uplus \dots \uplus \mathcal{D}(\tau_n).$$

Aucune entité n'est « sans tenant » ni « dans deux tenants à la fois ». Cette propriété est garantie dans le code par le **TenantFilter** de Spring Security, qui injecte le `tenant_id` dans chaque requête SQL via un `ThreadLocal`.

5.7.3 Exemple

Deux tenants sont inscrits : τ_1 (Auto-École Centrale) et τ_2 (Clinique Mélen). Les `ServiceRequests` associées sont :

$$\mathcal{D}(\tau_1) = \{sr_1, sr_2, sr_3\}, \quad \mathcal{D}(\tau_2) = \{sr_4, sr_5\}, \quad \mathcal{D}(\tau_1) \cap \mathcal{D}(\tau_2) = \emptyset.$$

Un acteur de τ_2 exécutant `GET /api/service-requests` ne verra jamais sr_1 , sr_2 ou sr_3 : le `TenantFilter` ajoute automatiquement `WHERE tenant_id = 'τ2'` à chaque requête.

5.8 Provisionnement d'un tenant : suite finie d'états

Ce modèle formalise l'inscription d'un nouveau tenant comme une suite d'états atomiques. Il est distinct de la machine à états des `ServiceRequests` (section 5.3.3) : il ne s'exécute qu'une seule fois par tenant, au moment de son inscription sur la plateforme.

5.8.1 Les sept états du provisionnement

Soit $\mathcal{S} = (S_0, S_1, \dots, S_6)$ la suite des états :

Table 5.2: Suite d'états du provisionnement d'un tenant

État	Nom	Description et précondition
S_0	Requête reçue	Réception de <code>POST /api/tenants {name, domain, sla}</code> .
S_1	Validation du domaine	Précondition : $\text{dom}_{\text{req}} \notin \{t.\text{domain} \mid t \in T\}$. Sinon : 409 <code>Conflict</code> .
S_2	Génération de l'id	Génération d'un $\text{id} \in \text{UUID}_4$ (collision $\approx 10^{-36}$).
S_3	Insertion en base	<code>INSERT INTO tenant</code> en transaction atomique. Rollback si échec.
S_4	Contexte d'isolation	Initialisation $\mathcal{D}(t_{\text{new}}) \leftarrow \emptyset$, <code>TenantFilter</code> configuré.
S_5	Génération JWT	Token signé (tenant_id , roles , exp), $\text{exp} = t_{\text{now}} + 24 \text{ h}$.
S_6	Tenant actif	$t_{\text{new}} \in T$ et $\mathcal{D}(t_{\text{new}}) = \emptyset$.

5.8.2 Propriété : injectivité du domaine

La précondition de S_1 garantit que la fonction $\text{domain} : T \rightarrow \text{Dom}$ est **injective** :

$$\text{domain}(t_i) = \text{domain}(t_j) \implies t_i = t_j.$$

Deux tenants distincts ne peuvent jamais partager le même domaine. Cette propriété complète les contraintes de cardinalité de la section 5.3 et est analogue à l'unicité des adresses IP sur un réseau : elle permet un routage sans ambiguïté par f_3 (section 5.6).

5.8.3 Exemple

Un gestionnaire soumet `POST /api/tenants {"domain":"cabinet-dupont"}`. En S_1 , le système vérifie `"cabinet-dupont" ∉ {t.domain | t ∈ T}` : précondition satisfaite. En S_2 , l'UUID

b9c2-4f1a-... est généré. En S_3 , la ligne est insérée de façon atomique. En S_4 , $\mathcal{D}(t_{\text{new}}) \leftarrow \emptyset$. En S_5 , le JWT initial est signé. En S_6 :

$$t_{\text{new}} \in T, \quad \mathcal{D}(t_{\text{new}}) = \emptyset, \quad \text{domain}(t_{\text{new}}) = \text{"cabinet-dupont"}.$$

Si une seconde requête tentait le même domaine, S_1 retournerait 409 **Conflict**, préservant l'injectivité. (chapitre 10).

CHAPITRE 6

Conception architecturale

La pile protocolaire en cinq couches qui structure la lecture de BCaaS a été présentée au chapitre 3 (figure 3.2). Le présent chapitre se concentre sur les choix d'architecture logicielle qui la concrétisent : l'architecture hexagonale, le monolithe modulaire et l'organisation en packages.

6.1 Architecture hexagonale et DDD

BCaaS adopte une architecture hexagonale (ports et adaptateurs) combinée au Domain-Driven Design. Le domaine métier (entités, règles, machines à états) est isolé au centre ; les détails techniques (persistance JPA, exposition REST, messagerie) sont relégués aux adaptateurs périphériques. Cette organisation garantit que le cœur métier ne dépend d'aucune technologie particulière, condition d'une évolution ultérieure sereine.

Concrètement, le flux d'une requête traverse les couches applicatives suivantes :

Controller (REST) → Service (domaine) → Repository (JPA) → Base de données,

les DTO assurant le découplage entre la représentation exposée et les entités persistées.

6.2 Monolithe modulaire « microservices-ready »

Conformément aux contraintes du sujet, BCaaS est un **monolithe modulaire** : les modules métier (acteurs, métiers, catalogues, demandes, factures, audit, analytique) sont organisés en packages cohérents et communiquent par appels en mémoire. Ce choix simplifie le déploiement et la coordination initiale tout en préservant des frontières nettes : les modules pourront, à terme, être extraits en microservices déployés indépendamment, sans réécrire le domaine (perspective détaillée au chapitre 13).

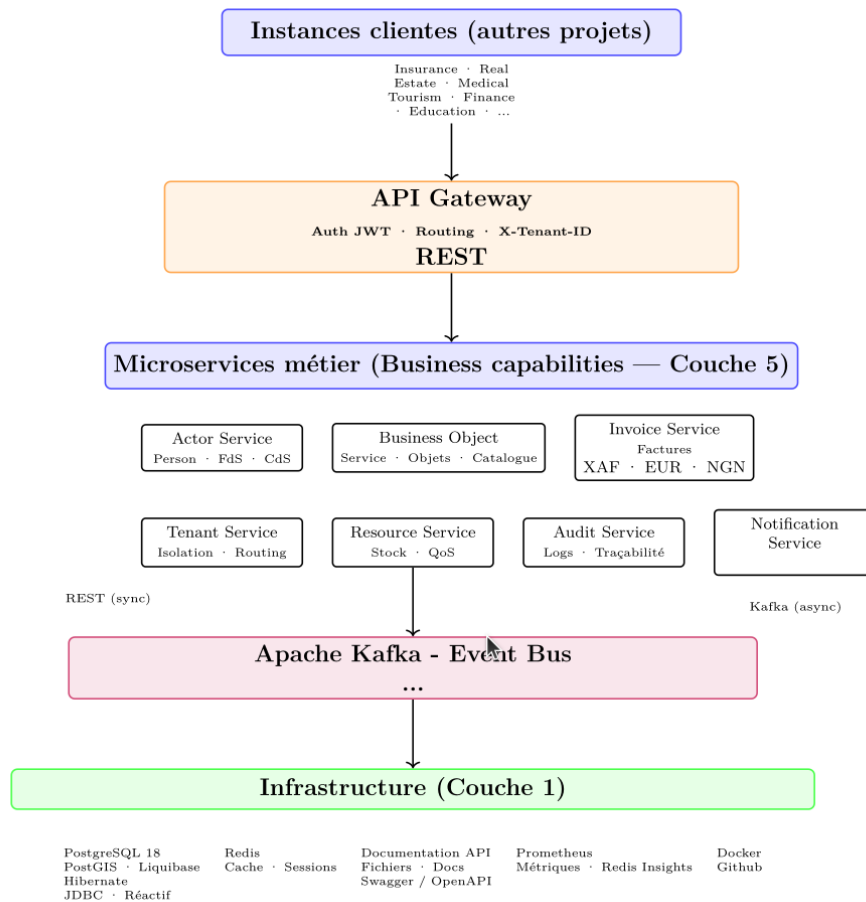


Figure 6.1: Architecture logique de BCaaS

6.3 Diagramme de packages

L'organisation en packages reflète les domaines fonctionnels : authentification, gestion des acteurs, gestion des métiers, gestion des services, gestion des médias, gestion des ressources et gestion des factures. Chaque package porte des responsabilités bien délimitées, ce qui facilite l'évolution indépendante des règles métier et des mécanismes

Tenant L'acteur Tenant suggère une architecture multi-tenant, où chaque client (organisation ou locataire) dispose de sa propre isolation des données et configurations. Ce package agit probablement comme un contexte global d'exécution permettant aux autres modules de fonctionner de manière isolée par tenant.

Authentification et gestion des acteurs Le package AUTHENTIFICATION gère les processus d'identification et de vérification des tenants. Il interagit directement avec GESTION DES ACTEURS qui centralise la définition, les rôles et les permissions des différentes entités utilisatrices (administrateurs, clients, fournisseurs, etc.). Cette séparation permet de dissocier le mécanisme technique d'authentification de la gestion administrative des profils.

Gestion des business et services Les packages GESTION DES BUSINESS et GESTION DES SERVICES couvrent le cœur métier. Le premier traite des règles propres à l'activité de l'entreprise, tandis que le second coordonne l'information, l'orchestration ou la souscription aux services proposés. Leur découpage facilite l'évolution indépendante des règles métier et des mécanismes de service.

Gestion des médias, ressources et factures Le package GESTION DES MEDIA est dédié aux contenus multimédias (images, vidéos, documents). GESTION DES RESSOURCES supervise les actifs techniques ou matériels nécessaires au fonctionnement (serveurs, licences, matériel, équipements). Enfin, GESTION DES FACTURES assure la chaîne de facturation, de la génération des factures au suivi des paiements.

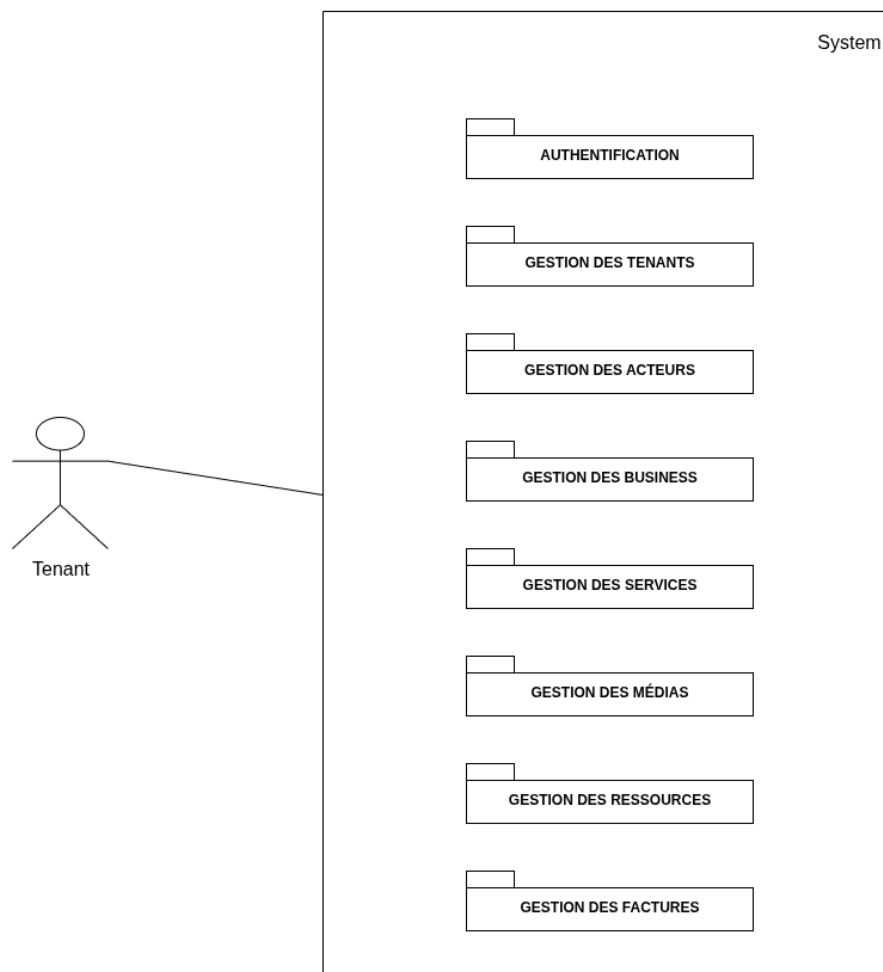


Figure 6.2: *Diagramme de packages de l'architecture du système*

CHAPITRE 7

Conception détaillée des données

Le défi central de BCaaS est de modéliser des entités métier hétérogènes (médecin, livre, appartement, produit agricole, etc.) avec un schéma de données unique et générique. Nous adoptons une modélisation Merise (MCD, MLD) complétée par un diagramme de classes UML.

7.1 Dictionnaire des entités

Le domaine compte quinze entités (plus l'énumération `SupportedCurrency` et la table de jonction `portfolio_businesses`). Le tableau 7.1 en récapitule la structure.

Table 7.1: *Entités du modèle de données — Business Core*

Entité (table)	Attributs principaux et relations
Tenant (tenants)	id, name (unique), domain, description, isActive, createdAt. Racine du multi-tenant.
User (users)	id, firstName, lastName, email (unique), password, phone, country, roles (Set), isActive, createdAt ; ManyToOne → Tenant.
Actor (actors)	id, role, bio, isActive, createdAt ; ManyToOne → User.
Business (businesses)	id, name, domain, description, typeOfInvolvedActors, requiredJobProfile, createdAt ; ManyToOne → Tenant.
BusinessRule (business_rules)	id, ruleKey, ruleValue, description, createdAt ; ManyToOne → Business.
ServiceCatalogue (service_catalogues)	id, name, description, basePrice, currency, isAvailable, createdAt ; ManyToOne → Business.
Resource (resources)	id, name, type, quantity, description, createdAt ; ManyToOne → Business.
Media (media)	id, type, url, label, createdAt ; ManyToOne → Business.
Portfolio (portfolios)	id, title, description, createdAt ; OneToOne → Actor ; ManyToMany → Business (portfolio_businesses).
ServiceRequest (service_requests)	id, serviceName, description, status, traceId, correlationId, 5 timestamps ; ManyToOne → Tenant, consumer Actor, provider Actor, ServiceCatalogue.
Invoice (invoices)	id, amount, currency, status, issuedAt, paidAt ; OneToOne → ServiceRequest (unique).
Message (messages)	id, content, isRead, createdAt ; ManyToOne → ServiceRequest, sender Actor, Tenant.
Notification (notifications)	id, title, message, isRead, createdAt ; ManyToOne → User, Tenant.
AuditEvent (audit_events)	id, entityType, action, previousStatus, newStatus, ipAddress, userAgent, metadata (JSON), tenantId, actorId, entityId, traceId (UUID bruts).

Toutes les entités utilisent une génération d'identifiant `GenerationType.UUID` et un `@PrePersist` initialisant `createdAt`.

7.2 Modèle Logique de Données (MLD)

Le passage au modèle relationnel traduit les associations par des clés étrangères. En particulier, l'isolation multi-tenant repose sur une colonne `tenant_id` (UUID, clé étrangère vers `tenants`) portée par les tables racines du tenant (`users`, `businesses`, `service_requests`, `messages`, `notifications`, `audit_events`). L'entité `Actor` n'a pas de `tenant_id` direct : elle hérite du contexte tenant via son `User`.

Table 7.2: *Entités du modèle de données – Business Core*

Table	Attributes
tenant	id_Tenant, Name, IsActive, CreatedAt
user	Id_User, FirstName, LastName, Email, Gender, Password, IsActive, Country, Phone, CreatedAt, #id_Tenant
actor	Id_Actor, NomProfessionnel, Speciality, bio, CreatedAt, Role, #Id_User
formation	Id_Formation, NomFormation, Description, Type_Formation, Niveau
media	Id_Media, Type, Url, Label, CreatedAt, #Id_Business
business	Id_Business, NameBusiness, Domain, Type_of_InvolvedActors, RequiredJobProfile, CreatedAt
business_rule	Id_BusinessRule, RuleKey, RuleValue, Description, CreatedAt
ressource	Id_Resource, NameRes, Type, Quantity, Description, CreatedAt, Disponibilité
service_catalogue	Id_SerCatalogue, NameSerCatalogue, Description, Currency, IsAvailable, CreatedAt
portfolio	Id_Portfolio, title, Description, Status, #Id_Business
service_request	Id_ServiceRequest, TraceId, ServiceName, Description, Status, #Id_Actor, #Id_Portfolio
message	id_Message, content, isRead, createdAt, #Id_ServiceRequest
notification	id_Notification, title, message, isRead, createdAt, #Id_ServiceRequest
invoice	id_Invoice, Amount, Currency, Status, UsedAt, PaidAt, IssueAt, #Id_ServiceRequest

7.3 Diagramme de classes métier

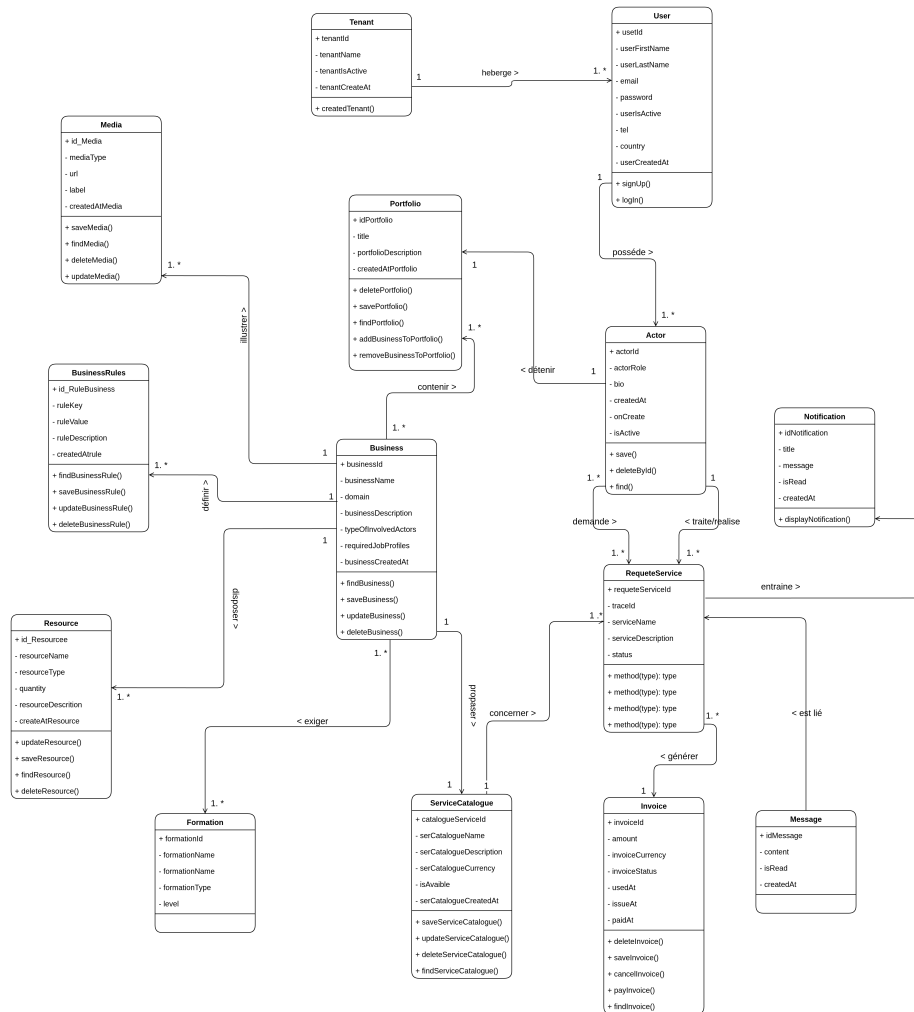


Figure 7.1: Diagramme de classes métier de BCaaS

7.4 Contraintes d'intégrité

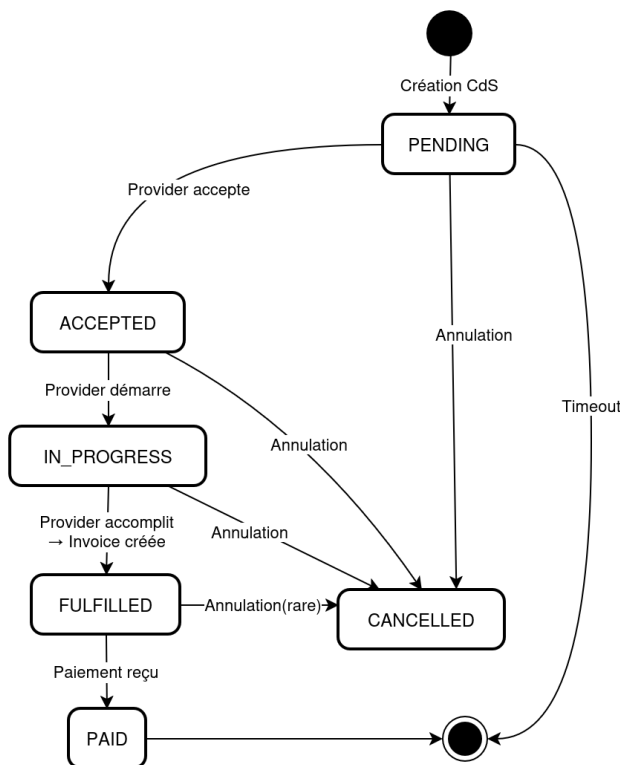
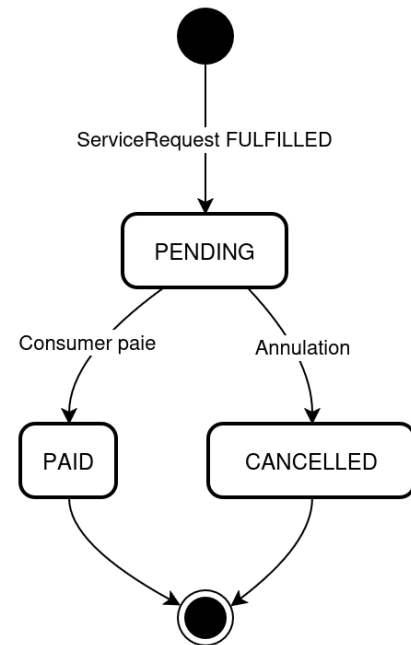
Le tableau 7.3 liste les principales contraintes d'intégrité référentielle et d'unicité.

Table 7.3: Principales contraintes d'intégrité

Table	Type	Description
tenants	Unique	Nom de tenant unique
users	Unique	Email unique
users	FK	<code>tenant_id</code> → tenants
portfolios	Unique + FK	Un portfolio par acteur
service_requests	FK	consumer, provider, catalogue, tenant existants
invoices	Unique + FK	Une facture par demande de service
messages, notifications, audit_events	FK (NOT NULL)	<code>tenant_id</code> obligatoire

7.5 Machines à états

Deux entités portent une machine à états explicite. La `ServiceRequest` suit le cycle `PENDING` → `ACCEPTED` → `IN_PROGRESS` → `FULFILLED`, avec `CANCELLED` accessible depuis tout état non terminal ; chaque transition horodate un champ dédié (`requestedAt`, `acceptedAt`, `startedAt`, `fulfilledAt`, `cancelledAt`). La transition `fulfill` déclenche la génération automatique de l'`Invoice`. La facture suit quant à elle `PENDING` → `PAID` ou `CANCELLED`.

**Figure 7.2:** Machine à états de *ServiceRequest***Figure 7.3:** Machine à états de *Invoice*

CHAPITRE 8

Implémentation technique

8.1 Pile technologique

Le tableau 8.1 présente la pile effectivement utilisée. Conformément au cadrage MVP retenu, les versions et composants y sont indiqués tels qu'ils sont réellement mis en œuvre ; les briques avancées non câblées sont reportées au chapitre 13.

Table 8.1: *Pile technologique de BCaaS*

Composant	Technologie	Version
Framework backend	Spring Boot (servlet, MVC)	3.5.x
Langage	Java	21 LTS
Base de données	PostgreSQL	16
Migrations	Liquibase	4.x
Sécurité	Spring Security + JWT (jjwt)	0.12.6
Documentation API	SpringDoc OpenAPI	2.8.6
Cache / pub-sub	Redis	7
Bus d'événements	Apache Kafka (infra)	7.6 (image)
Build	Maven	3.9+
Tests	JUnit 5 + Mockito + AssertJ	152 tests
Conteneurisation	Docker + Docker Compose	—
Frontend	Next.js / React / TypeScript	16 / 19

8.2 Structure du projet

Le projet suit l'architecture hexagonale combinée au DDD. L'arborescence backend est organisée par responsabilités techniques au sein du package `com.bizcore.bizcore_backend`.

```
1 src/main/java/com/bizcore/bizcore_backend/
2   +-- auth/           # Authentification (register, login, roles)
3   +-- config/         # Configurations (Kafka, Redis, Security, cache)
4   +-- controller/     # Controllers REST
5   +-- domain/         # Entites JPA (15) + enum SupportedCurrency
6   +-- dto/            # Objets de transfert
7   +-- exception/      # GlobalExceptionHandler, ApiError
8   +-- listener/       # Listener Kafka (consommateur)
9   +-- repository/     # Repositories Spring Data (tenant-aware)
10  +-- security/        # JwtService, JwtAuthFilter, TenantFilter, TenantContext
```

```

11  +-- service/          # Services metier
12  src/main/resources/
13  +-- application.properties
14  +-- db/changelog/ # Migrations Liquibase
15  src/test/            # 16 classes, 152 tests (JUnit 5 + Mockito)

```

Listing 8.1: Structure du projet backend

8.3 Multi-tenant et isolation

L'isolation repose sur le modèle « schéma partagé, colonne `tenant_id` » et sur trois pièces coordonnées.

1. TenantContext — un ThreadLocal. Chaque requête HTTP dispose d'une copie isolée du tenant actif, stockée dans un `ThreadLocal<UUID>`, ce qui évite toute fuite entre requêtes concurrentes partageant le même pool de threads.

2. TenantFilter — extraction depuis le JWT. Filtre `OncePerRequestFilter` d'ordre 2 (après le filtre d'authentification), il lit le claim `tenantId` du JWT et le place dans le contexte, puis le purge dans un bloc `finally`.

```

1  String tenantIdStr = jwtService.extractTenantId(jwt);    // claim "tenantId"
2  TenantContext.setTenantId(UUID.fromString(tenantIdStr));
3  try {
4      filterChain.doFilter(request, response);
5  } finally {
6      TenantContext.clear();    // pas de fuite inter-tenant
7  }

```

Listing 8.2: Extrait simplifié de `TenantFilter`

3. Injection du claim à la connexion. À l'authentification, `JwtService` embarque `tenantId` et `tenantName` dans le token. Le tenant est ainsi dérivé de l'utilisateur (colonne `users.tenant_id`) et relu à chaque requête sans appel supplémentaire à la base. L'isolation est appliquée dans la couche service via des requêtes *tenant-aware* de type `findAllByTenantId(...)`.

Garantie d'isolation et limite assumée

Le `tenant_id` circule dans le contexte d'exécution et conditionne les requêtes des services principaux ; un générateur de clés de cache préfixe par tenant. **Limite assumée :** l'isolation n'est pas une barrière globale (ni filtre Hibernate `@Filter`, ni *Row-Level Security* PostgreSQL) ; elle repose sur la discipline de chaque méthode de service. Le renforcement de cette barrière est traité au chapitre 13.

8.4 Sécurité : JWT et RBAC

L'authentification repose sur un couple email / mot de passe chiffré par BCrypt. Le token JWT, signé en HMAC-SHA, porte les rôles et le `tenantId`. Le contrôle d'accès est basé sur les rôles (`USER`, `ADMIN`, `PROVIDER`, `CONSUMER`), un même utilisateur pouvant cumuler plusieurs rôles. Spring Security joue ici le rôle d'un pare-feu applicatif filtrant les requêtes selon l'analogie du tableau 3.1.

8.5 Persistance et migrations

Le schéma est géré par Liquibase, garantissant des migrations versionnées et reproductibles (création du schéma, ajout des tables `notifications`, `messages`, `audit_events`, colonnes `tenant_id` contraintes par clé étrangère et indexées). Le paramètre `ddl-auto=none` laisse Liquibase seul maître du schéma en exécution.

8.6 Temps réel et événementiel

La messagerie instantanée sur les demandes de service est réalisée par **Redis publish/-subscribe** : le service de messages publie sur un canal `messages:<requestId>`. Un **bus d'événements Kafka** est configuré (`KafkaTemplate` côté producteur, `@KafkaListener` sur le topic `bizcore.events` côté consommateur) ; l'infrastructure est posée et le consommateur opérationnel, la *production* d'événements restant à câbler (perspective au chapitre 13). Le cache Redis est de même configuré (gestionnaire de cache, TTL par espace) et prêt à être activé par annotations.

8.7 Documentation et conteneurisation

L'API est documentée automatiquement via SpringDoc OpenAPI et explorable par Swagger UI (annexe A.3). Un `docker-compose` décrit l'environnement local complet (application, PostgreSQL, Redis, Kafka avec *healthchecks* et dépendances ordonnancées) ; en production, l'API et sa base s'exécutent sur Render.

CHAPITRE 9

Protocole d'échange pour un cas métier concret

Pour illustrer la « pensée protocolaire » du projet, ce chapitre formalise l'échange d'un cas métier complet — la création d'un **Business** puis l'ajout d'un service à son catalogue — comme on spécifierait un protocole réseau : en-têtes obligatoires, payload, réponses, codes d'erreur et sémantique de transport.

9.1 Cas choisi : création d'un Business et ajout au catalogue

Ce cas correspond à deux requêtes distinctes mais liées, typiques de l'enregistrement d'une nouvelle activité (auto-école, cabinet de conseil, etc.) dans un tenant.

9.2 Protocole : création d'un Business

Endpoint : POST /api/businesses

En-têtes obligatoires :

- **Authorization**: Bearer <JWT> — authentification (analogie : ticket de session) ; le claim `tenantId` joue le rôle de l'adresse réseau.

```
1 {
2   {
3     "name": "Auto-Ecole Centrale",
4     "domain": "driving_school",
5     "description": "Numero 1 dans la formation auto",
6     "typeOfInvolvedActors": ["instructor", "student"],
7     "requiredJobProfile": "moniteur",
8     "tenantId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
9   }
}
```

Listing 9.1: Corps de la requête (payload)

```
1 {
2   "id": "550e8400-e29b-41d4-a716-446655440000",
3   "name": "Auto-Ecole Centrale",
4   "domain": "driving_school",
5   "createdAt": "2025-05-04T10:00:00Z"
6 }
```

Listing 9.2: Réponse (succès 201 Created)

Codes d'erreur (analogie : messages ICMP).

- 400 Bad Request : payload invalide ;
- 401 Unauthorized : JWT manquant ou invalide ;
- 403 Forbidden : rôle insuffisant ;
- 404 Not Found : ressource inexistante ;
- 409 Conflict : ressource déjà existante.

9.3 Protocole : ajout au catalogue de services

Endpoint : POST /api/service-catalogues/business/{businessId}

En-têtes : identiques, plus une corrélation optionnelle.

```
1 {  
2   "name": "Cours de conduite",  
3   "description": "Formation theorique et pratique de 3 mois",  
4   "basePrice": 150000,  
5   "currency": "XAF",  
6   "isAvailable": true  
7 }
```

Listing 9.3: Corps de la requête

Sémantique du transport.

- **Mode** : synchrone (REST) — analogie TCP (fiable, ordonné) ;
- **Timeout** : de l'ordre de 30 secondes ;
- **Idempotence** : non (chaque POST crée une nouvelle ressource) ;
- **Retry** : recommandé avec *backoff* exponentiel (au plus 3 tentatives).

CHAPITRE 10

Traçabilité et observabilité

Dans une architecture multi-tenant et distribuée comme la plateforme BCaaS, l'absence de visibilité sur l'état interne du système constitue un risque majeur. Pour garantir une transparence totale sur le cycle de vie des transactions, ce chapitre formalise notre couche d'observabilité.

En transposant les techniques de supervision et d'analyse de protocoles propres aux infrastructures réseaux — à l'image d'une capture de flux ou d'une analyse de paquets sous Wireshark —, nous articulons ce module autour d'éléments réellement implémentés au cœur de l'application :

1. **Une piste d'audit** : Matérialisée par l'entité `AuditEvent`, elle enregistre de manière exhaustive les actions clés, les adresses IP et les dimensions de sécurité de chaque transaction.
2. **Un système de tracing distribué** : L'utilisation des identifiants `traceId` et `correlationId` (UUID) qui se propagent dans tout le système pour suivre les flux de bout en bout, jouant le rôle des numéros de séquence TCP.
3. **Un service analytique** : Un module dédié exposé par API pour calculer les indicateurs clés (KPIs) des tenants, calqués sur les métriques réseaux fondamentales (latence moyenne, débit, taux d'erreur).

Ce dispositif se trouve complété en fin de chapitre par une proposition d'intégration de logs structurés au format JSON.

10.1 Piste d'audit

L'entité `AuditEvent` enregistre les événements significatifs du système : type d'entité concernée (`SERVICE_REQUEST`, `INVOICE`, `ACTOR`, `BUSINESS`, `SERVICE_CATALOGUE`), action (`CREATED`, `STATUS_CHANGED`, `UPDATED`, `DELETED`), statut précédent et nouveau, adresse IP, *user agent* et métadonnées au format JSON.

Conçue selon un principe de stricte **immuabilité**, la piste d'audit fonctionne exclusivement en mode *Append-Only* (écriture seule) : aucune opération de mise à jour (`UPDATE`) ou de suppression (`DELETE`) n'est exposée par l'API ou autorisée par le système, garantissant la non-répudiation des logs même en cas de compromission d'un administrateur de tenant.

L'enregistrement capture les dimensions clés de chaque transaction :

- **Le contexte d'application** : Le type d'entité concernée.
- **La nature de l'opération** : L'action effectuée, accompagnée de l'état précédent et du nouvel état de l'entité.

- **L’empreinte d’infrastructure** : L’adresse IP de l’appelant ainsi que son *user agent* (navigateur ou client applicatif).
- **La charge utile** : Des métadonnées dynamiques stockées dans un champ au format JSONB sous PostgreSQL. Ce choix technique permet de supporter un polymorphisme des données métiers (les métadonnées d’une facture différant de celles d’un catalogue) sans altérer le schéma relationnel.

Chaque événement porte obligatoirement les identifiants bruts `tenantId`, `actorId`, `entityId` et `traceId`. L’API d’audit permet d’interroger et de filtrer ces événements par demande de service, par acteur, par tenant et par trace.

Analogie : Deep Packet Inspection et Sonde Réseau

La capture des données d’audit est réalisée de manière non intrusive via un intercepteur applicatif (`OncePerRequestFilter` au niveau de la couche de sécurité), agissant comme une sonde réseau (SPAN/Mirroring). L’analyse de la charge utile JSONB est analogue au *Deep Packet Inspection* (DPI), où l’on inspecte non seulement les en-têtes du paquet (IP, Tenant, Acteur), mais également le contenu profond de la charge utile applicative pour en valider l’intégrité opérationnelle.

10.2 Tracing : `traceId` et `correlationId`

Chaque `ServiceRequest` véhicule de manière intrinsèque un `traceId` et un `correlationId` sous forme d’identifiants uniques universels (UUID). Bien qu’ils concourent tous deux à la visibilité du système, ils répondent à des granularités distinctes :

- **`traceId`** : Il est associé à une unique requête HTTP entrante. Il permet de suivre le fil d’exécution synchrone d’un bout à l’autre des couches logicielles (*Controller*, *Service*, *Repository*) pour une opération donnée.
- **`correlationId`** : Il englobe une opération métier globale pouvant impliquer plusieurs requêtes ou événements asynchrones distincts (par exemple, une demande de service qui déclenche une génération de facture puis une notification). Toutes ces sous-opérations partagent le même `correlationId`, permettant de lier logiquement des transactions hétérogènes.

Le mécanisme repose sur la **propagation de contexte**. À l’entrée du système, la passerelle applicative gère la genèse de ces identifiants et les injecte dans les en-têtes HTTP (`X-Trace-Id` et `X-Correlation-Id`). Les services internes extraient ces en-têtes et les rattachent au fil d’exécution via le mécanisme MDC (*Mapped Diagnostic Context*) de la brique de journalisation, garantissant leur inscription systématique dans chaque log généré.

Analogie : En-tête de protocole et circuit virtuel

Le `traceId` et le `correlationId` agissent exactement comme les en-têtes de contrôle d'un protocole réseau (tels que les numéros de séquence TCP ou les étiquettes MPLS). Au lieu d'altérer la charge utile de la requête métier, ces identifiants sont encapsulés dans les couches de transport applicatif (en-têtes HTTP), permettant aux routeurs applicatifs et aux commutateurs de logs de corrélérer et de reconstituer la topologie complète du flux de données sans inspecter le contenu sémantique du message.

10.3 Analytique et KPIs

Le service d'analytique calcule en temps réel, pour chaque tenant, les indicateurs clés de performance (KPIs) formalisés au chapitre 5. Afin de préserver les performances du noyau transactionnel et d'éviter des requêtes d'agrégation (`COUNT`, `SUM`) lourdes sur la base de données relationnelle, le système implémente un principe de **séparation des responsabilités** (CQRS applicatif). Les métriques sont soit calculées de manière asynchrone au fil de l'eau via le bus d'événements, soit agrégées et matérialisées dans un cache en mémoire à haute disponibilité (`Redis`).

L'endpoint `GET /api/analytics/tenants/{tenantId}/kpis` expose ces indicateurs. Pour garantir une haute réactivité, les réponses de cet endpoint sont associées à une durée de vie stricte (*TTL - Time To Live*), limitant la fréquence de recalcul. Le tableau 10.1 dresse la liste de ces métriques opérationnelles et explicite leur équivalent conceptuel dans le domaine des réseaux.

Table 10.1: Métriques de BCaaS et leur analogie réseau

Métrique	Analogie réseau	Calcul
Taux de conversion	Taux de livraison de paquets	demandes finalisées / total
Latence moyenne	RTT (Round Trip Time)	temps moyen de traitement
Débit (req/s)	Bande passante	requêtes par minute
Taux d'erreur	Taux de perte	réponses 4xx/5xx / total
Panier moyen	Taille moyenne de payload	chiffre d'affaires / nb factures
Délai d'exécution	Délai de routage	temps requête → finalisation

Analogie : Cache DNS et TTL

Le mécanisme de mise en cache des KPIs avec un TTL (Time To Live) est rigoureusement identique à la gestion d'un cache DNS ou d'une table de routage dynamique. Un équipement réseau ne sollicite pas le serveur racine pour chaque paquet à acheminer ; il maintient une copie locale des routes et des résolutions de noms pendant une durée déterminée afin de maximiser le débit global et de minimiser la surcharge CPU du système.

10.4 Logs structurés (proposition)

Pour compléter le dispositif d'observabilité, une stratégie de journalisation structurée au format JSON est proposée. Contrairement aux chaînes de caractères de texte brut traditionnelles (*Plain Text*) dont l'analyse logicielle requiert des expressions régulières complexes et fragiles, le format JSON standardise nativement la sémantique de chaque ligne de log. Chaque entrée intègre systématiquement le contexte d'exécution complet : `tenant_id`, `trace_id`, `actor_id`, le nom du service, l'opération exécutée, la durée exacte du traitement et le code de statut HTTP retourné.

L'implémentation de cette proposition repose sur un découplage total : l'application écrit ces objets JSON directement sur la sortie standard (STDOUT), minimisant l'impact sur le thread d'exécution métier. Un agent de collecte léger (tel que *Promtail* ou *FluentBit*) se charge ensuite d'aspirer ces flux de manière asynchrone pour alimenter une chaîne de centralisation (*Grafana Loki* ou pile *ELK*). Cette structuration permet d'effectuer des requêtes multi-critères instantanées et de corréler une alerte de performance globale avec les logs spécifiques d'un tenant.

Analogie : Protocoles d'export de flux (NetFlow / IPFIX)

La journalisation structurée en JSON is l'équivalent applicatif des protocoles de métrologie réseau tels que *NetFlow* (Cisco) ou *IPFIX*. Un routeur réseau n'exporte pas des lignes de texte brut pour décrire le trafic ; il transmet des enregistrements structurés contenant des champs normalisés (adresses IP, ports, drapeaux TCP, volumes). Cette standardisation permet aux collecteurs réseau, tout comme à nos indexeurs de logs, d'agréger, de filtrer et de cartographier le trafic en temps réel à grande échelle.

CHAPITRE 11

Frontend et déploiement

11.1 Architecture du frontend

Le frontend est développé avec Next.js (App Router) et React en TypeScript. L'architecture de l'interface se structure autour d'un site vitrine public, exposant l'approche protocolaire, la documentation interactive de la plateforme et les guides de démarrage. Un tableau de bord d'administration, permettant aux acteurs authentifiés de piloter les demandes de service et de gérer les acteurs, est prévu dans une version future.

11.2 Choix techniques

Le choix de Next.js est motivé par sa capacité à combiner le rendu côté serveur (*Server-Side Rendering* - SSR) pour la sécurisation des tableaux de bord dynamiques, et la génération statique (*Static Site Generation* - SSG) pour maximiser les performances de chargement du site vitrine. Le tableau 11.1 dresse la cartographie logicielle de l'interface et ses justifications.

Table 11.1: *Technologies du frontend et leur rôle architectural*

Besoin	Technologie	Justification
Framework	Next.js 16 (App Router)	SSR/SSG, optimisation du TTFB, routage par dossier
UI	React 19 + TypeScript	Composants typés, gestion robuste des états d'affichage
Styling	Tailwind CSS 3	Conception atomique, design système fluide
Composants	Radix UI	Primitives de composants hautement accessibles
Animations / 3D	Framer Motion, react-three/fiber	Restitution immersive de la métaphore réseau
État global	Zustand	Store centralisé léger, découplé du cycle de rendu React
Données serveur	TanStack Query + Axios	Gestion du cache local et synchronisation asynchrone avec l'API

La couche de communication du frontend est construite avec Axios et encapsulée dans TanStack Query pour la synchronisation des données. Un intercepteur Axios injecte automatiquement le jeton JWT d'authentification sur chaque requête sortante. L'injection des en-têtes de tracing distribué (*X-Trace-Id*, *X-Correlation-Id*) est prévue dans une version ultérieure.

11.3 Pages principales

Les routes implémentées au sein de l'application comprennent :

- **Espace public** : La page d'accueil (/), la démonstration interactive du protocole (/demo), l'assistant conversationnel d'assistance (/chat), la documentation technique (/docs, /docs/[id]) et les guides d'intégration (/guides, /guides/[slug]).
- **Espace d'administration (à venir)** : Tableau de bord central (/dashboard) et sous-pages spécialisées : gestion des demandes (/dashboard/requests), des acteurs (/dashboard/actors), des règles métiers (/dashboard/businesses), de la facturation (/dashboard/invoices), des catalogues de services (/dashboard/services) et des paramètres globaux (/dashboard/settings). Les ébauches de ces pages sont présentes dans le code mais leur intégration fonctionnelle complète est repoussée à une version ultérieure.

11.4 Déploiement

L'infrastructure applicative adopte une topologie de déploiement multi-cloud distribuée pour optimiser les coûts et isoler les responsabilités d'exécution :

- **Couche Backend et Persistance** : L'API Spring Boot et la base de données relationnelle PostgreSQL sont déployées sur la plateforme **Render** (<https://bizcore-api.onrender.com>). La documentation interactive des points d'accès (Swagger UI) y est directement hébergée et accessible pour les tests d'intégration.
- **Couche Frontend** : Le serveur de rendu Next.js et l'interface utilisateur sont déployés sur la plateforme **Vercel** (<https://bizcore-liard.vercel.app>), bénéficiant nativement d'un réseau de diffusion de contenu (CDN) mondial pour minimiser la latence réseau lors du téléchargement des ressources statiques.

La sécurité des communications entre ces deux environnements distants est assurée par le chiffrement systématique des flux via le protocole TLS (HTTPS) et par la configuration d'une politique de filtrage de ressources partagées (CORS) stricte sur l'API Spring Boot, restreignant les accès légitimes au seul domaine du frontend.

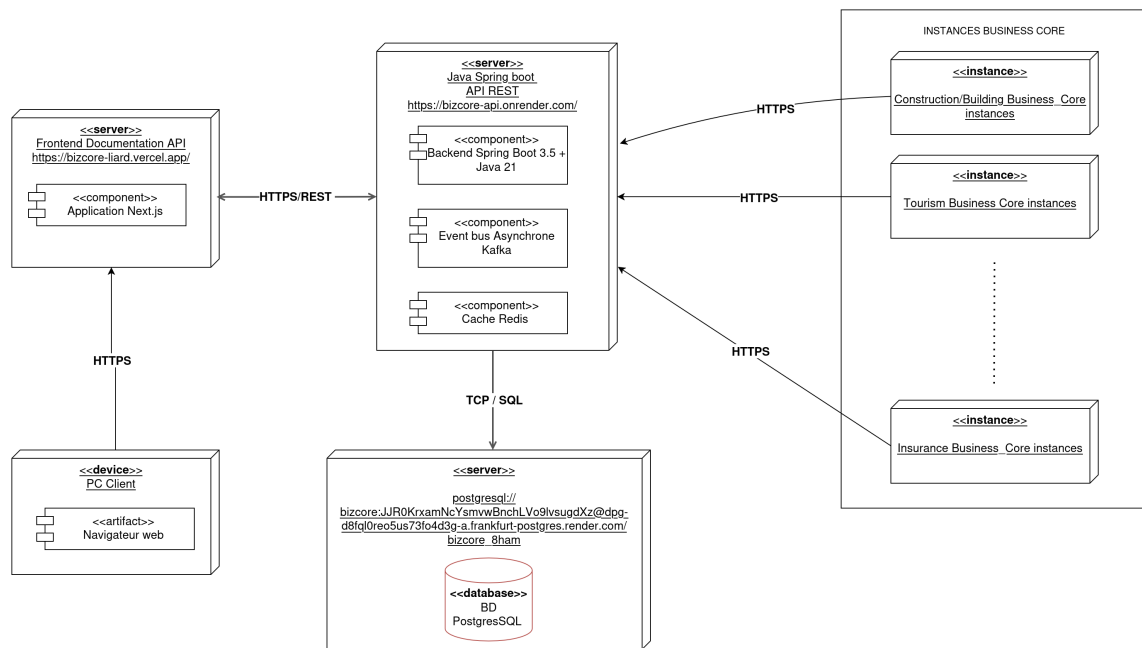


Figure 11.1: Diagramme de déploiement du système

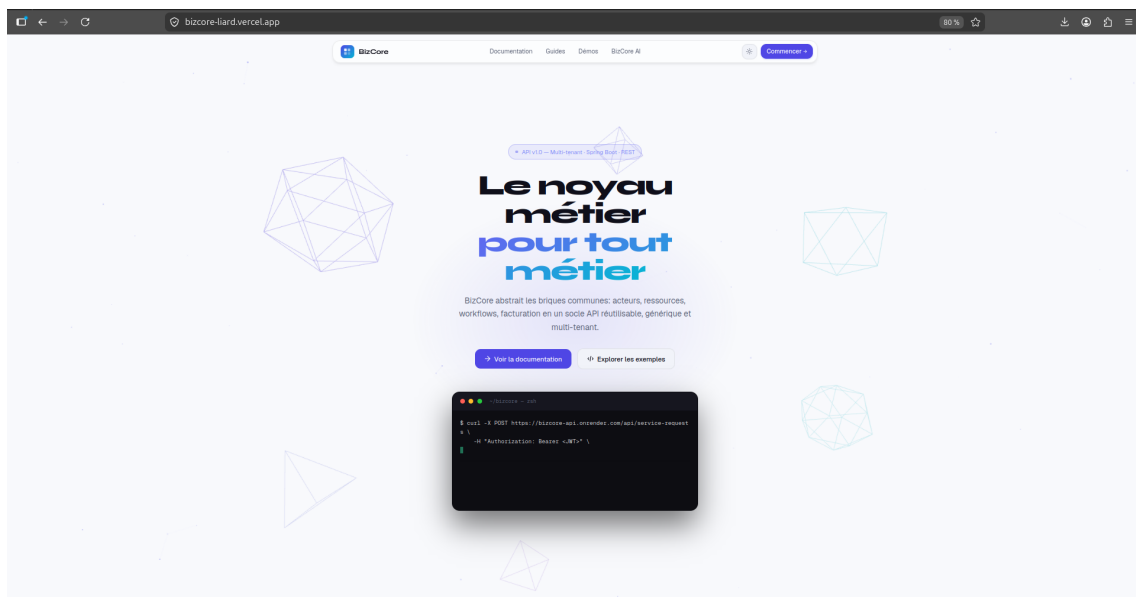


Figure 11.2: Page d'accueil du site vitrine déployé sur Vercel

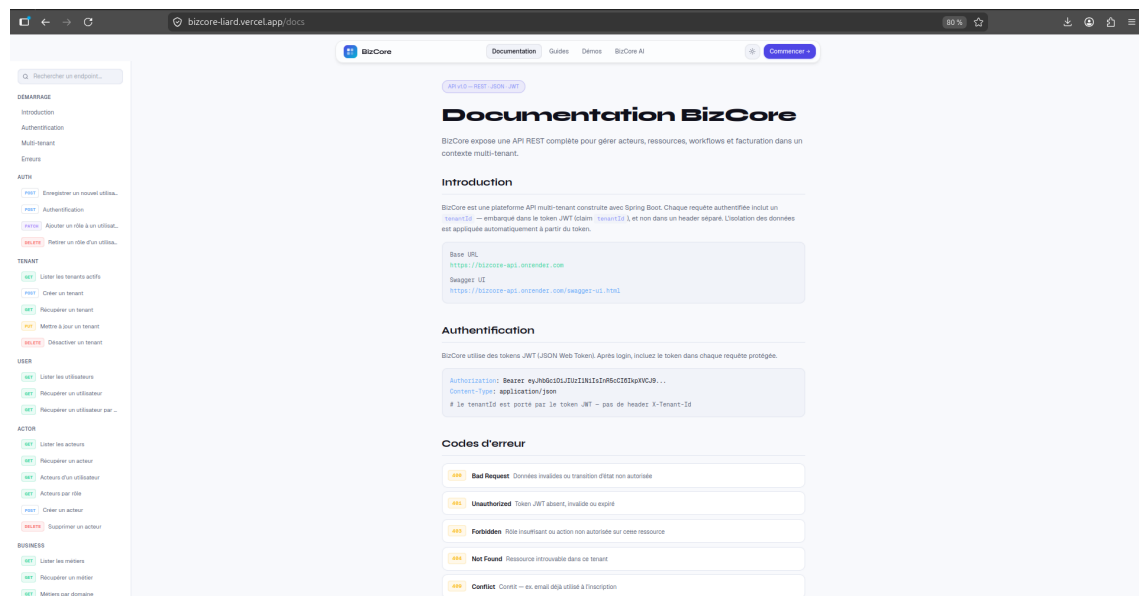


Figure 11.3: Documentation interactive de l'API

CHAPITRE 12

Tests et validation

La qualité d'un système logiciel ne se décrète pas : elle se démontre. Ce chapitre présente l'ensemble de la démarche de validation mise en œuvre pour BCaaS, depuis la stratégie globale de test jusqu'au scénario de validation bout-en-bout, en passant par les critères d'acceptation formels. L'objectif est de montrer que chaque brique du système — sécurité, logique métier, isolation multi-tenant, génération de factures — a été vérifiée de manière systématique et reproductible.

12.1 Stratégie de test

12.1.1 Vue d'ensemble et outillage

La validation de BCaaS repose sur **16 classes de test** totalisant **152 méthodes**, organisées selon une pyramide de test à trois niveaux. L'ensemble du dispositif s'appuie sur la stack de test standard de l'écosystème Spring Boot :

JUnit 5 (*Jupiter*) Socle d'exécution des tests : annotations `@Test`, `@BeforeEach`, `@AfterEach`, `@ParameterizedTest`. JUnit 5 apporte la prise en charge des extensions (`@ExtendWith`), utilisée notamment pour injecter le contexte Spring ou Mockito.

Mockito Bibliothèque de création de *mocks* et de *stubs*. Elle permet de simuler les dépendances externes d'un composant (repositories, services tiers) afin de tester la logique d'un service en totale isolation, sans base de données réelle ni appel réseau.

AssertJ Bibliothèque d'assertions fluides (`assertThat(...).isEqualTo(...)`, `.contains(...)`, `.isNotNull(...)`). Sa syntaxe chaînée améliore la lisibilité des assertions et produit des messages d'erreur plus explicites que les assertions JUnit natives.

spring-boot-starter-test Méta-dépendance qui embarque JUnit 5, Mockito et AssertJ, ainsi que `MockMvc` pour les tests de contrôleurs HTTP et `@SpringBootTest` pour les tests d'intégration avec contexte Spring complet.

spring-security-test Extension dédiée aux tests de la couche de sécurité Spring Security. Elle fournit les annotations `@WithMockUser` et `@WithUserDetails`, ainsi que les utilitaires `SecurityMockMvcRequestPostProcessors` pour simuler des requêtes authentifiées sans générer de vrais tokens JWT.

12.1.2 Environnement de test

Les tests s'exécutent dans un environnement entièrement isolé de la base de données de production ou de développement. Deux choix techniques fondamentaux garantissent cette isolation :

Base H2 en mémoire. H2 est un moteur de base de données relationnelle embarqué, écrit en Java, qui s'exécute entièrement en mémoire vive (*in-memory*). Il n'est pas nécessaire d'installer ni de démarrer un serveur de base de données externe : H2 démarre automatiquement avec le contexte Spring et s'arrête à la fin de la suite de tests. La configuration `ddl-auto=create-drop` garantit que le schéma est recréé à chaque lancement et détruit à la fin, rendant chaque exécution de tests parfaitement reproductible et indépendante de l'état laissé par une exécution précédente.

Liquibase désactivé. En production et en développement, BCaaS utilise Liquibase pour gérer les migrations de schéma de façon versionnée. Liquibase est désactivé dans le profil de test : le schéma est entièrement reconstruit par Hibernate à partir des entités JPA (`create-drop`), ce qui garantit que les tests ne dépendent pas d'un état de migration particulier et s'exécutent aussi bien sur la première version que sur une version ultérieure du schéma.

Profil Spring dédié. Un profil Spring `test` est activé pour les tests d'intégration. Ce profil surcharge les propriétés de connexion (`spring.datasource.url=jdbc:h2:mem:testdb`), désactive les mécanismes de cache, et configure des secrets JWT simplifiés adaptés à l'environnement de test.

12.1.3 Les trois niveaux de test

La stratégie de test de BCaaS suit le principe de la **pyramide de test** : une large base de tests unitaires rapides, une couche intermédiaire de tests de contrôleurs, et un sommet de tests d'intégration plus lents mais plus représentatifs du comportement réel du système.

Niveau 1 — Tests unitaires de services

Les tests unitaires ciblent la **logique métier pure** des services applicatifs, en isolation complète de toute infrastructure. Toutes les dépendances (repositories, services tiers) sont remplacées par des mocks Mockito.

Les services couverts sont :

- **InvoiceService** : génération automatique de facture lors de la transition `fulfill`, calcul des montants, association à la demande de service et au tenant.
- **BusinessService** : création, modification et suppression de métiers ; vérification des règles de nommage et d'unicité au sein d'un tenant.
- **TenantService** : provisionnement d'un nouveau tenant, vérification de l'absence de doublon, initialisation des configurations par défaut.

- **ActorService** : gestion du cycle de vie des acteurs (prestataires, clients) et de leurs rôles au sein d'un tenant.
- **ServiceRequestService** : machine à états de la demande de service ; validation des transitions autorisées et déclenchement des effets de bord (notification, facturation).
- **ServiceCatalogueService** : gestion des services offerts par un métier ; règles de visibilité et de tarification.
- **BusinessRuleService** : évaluation des règles métier configurables par tenant ; vérification de la conformité des demandes aux politiques en vigueur.
- **TenantContextTest** : vérification du bon fonctionnement du mécanisme de propagation du `tenant_id` dans le contexte de thread (`ThreadLocal`) ; test des cas de corruption ou d'absence de contexte.

Niveau 2 — Tests de contrôleurs (WebMvc)

Les tests de contrôleurs utilisent `MockMvc`, qui simule des requêtes HTTP sans démarrer un vrai serveur web. Le contexte Spring est chargé partiellement (`@WebMvcTest`), incluant uniquement la couche web (contrôleurs, filtres, sérialiseurs JSON) et excluant la couche de persistance.

Les services sont mockés via `@MockBean`, ce qui permet de se concentrer sur :

- la **désérialisation** des corps de requêtes JSON et la validation des contraintes (`@Valid`, `@NotNull`, tailles, formats) ;
- le **routage** des URLs vers les bonnes méthodes de contrôleur ;
- la **sérialisation** des réponses JSON et les codes HTTP retournés (200, 201, 400, 403, 404, 409) ;
- la **gestion des erreurs** via les `@ExceptionHandler` et le `ControllerAdvice` global.

Les contrôleurs couverts sont **BusinessController**, **InvoiceController**, **ActorController** et **ServiceRequestController**.

Niveau 3 — Tests de sécurité et d'intégration

Ce niveau constitue le sommet de la pyramide : les tests chargent le contexte Spring complet (`@SpringBootTest`) avec la base H2 en mémoire, et exercent le système de bout en bout à travers plusieurs couches.

JwtServiceTest Vérifie la génération, la signature et la validation des tokens JWT : format du token, présence et valeur des *claims* (`sub`, `tenantId`, `exp`, `iat`), rejet des tokens expirés ou falsifiés, et extraction correcte du `tenantId` depuis le token.

SecurityIntegrationTest Teste le filtre de sécurité sur l'ensemble des endpoints protégés : rejet des requêtes sans token (401), rejet des tokens invalides (401), autorisation correcte des requêtes avec un token valide, et vérification que les endpoints publics (inscription, connexion) sont bien accessibles sans authentification.

ServiceRequestIntegrationTest Scénario d'intégration complet de la demande de service : création d'un contexte tenant, d'un acteur demandeur et d'un prestataire, d'un service au catalogue, émission d'une demande, puis parcours de toutes les transitions de la machine à états (`pending` → `accepted` → `in_progress` → `fulfilled`) avec vérification, à chaque étape, de l'état persisté en base et de la génération automatique de la facture lors du passage à `fulfilled`.

12.2 Critères d'acceptation

12.2.1 Définition et méthode

Les critères d'acceptation formalisent les conditions minimales que BCaaS doit satisfaire pour être considéré comme livrable. Chaque critère est identifié par un code **ACC-XXX**, formulé de manière vérifiable et associé à une méthode de vérification concrète. Cette approche s'inspire des pratiques de *Behavior-Driven Development* (BDD) et garantit que la validation n'est pas subjective : un critère est soit satisfait, soit non satisfait, sans ambiguïté.

12.2.2 Tableau des critères

Table 12.1: Critères d'acceptation et méthodes de vérification

ID	Critère	Méthode de vérification
ACC-001	Les endpoints REST répondent correctement aux requêtes valides et retournent les codes HTTP appropriés (2xx, 4xx) pour les cas d'erreur.	Tests WebMvc + validation manuelle via Swagger UI
ACC-002	L'authentification par JWT fonctionne : génération, signature, validation et rejet des tokens expirés ou malformés.	<code>JwtServiceTest</code> , <code>SecurityIntegrationTest</code>
ACC-003	Le cycle complet CdS → FdS (demande de service à facture) se déroule sans erreur à travers toutes les transitions de la machine à états.	<code>ServiceRequestIntegrationTest</code>
ACC-004	La facture est générée automatiquement lors du passage à l'état <code>fulfilled</code> , avec les montants et références corrects.	Test de service <code>InvoiceServiceTest</code> + vérification en base H2
ACC-005	L'isolation multi-tenant est garantie : un tenant ne peut accéder aux données d'un autre, même en cas de manipulation du token.	<code>TenantContextTest</code> + <code>SecurityIntegrationTest</code>
ACC-006	La documentation Swagger (OpenAPI) est à jour et reflète fidèlement tous les endpoints exposés.	Validation visuelle sur <code>/swagger-ui.html</code>

12.2.3 Analyse des critères

ACC-001 — Endpoints REST. Ce critère couvre la surface d'API complète de BCaaS. Les tests WebMvc vérifient systématiquement les cas nominaux (requête valide → réponse 200/201) et les cas d'erreur les plus fréquents : corps de requête invalide (400), ressource inexistante (404), conflit d'identifiant (409), et accès non autorisé (403). La validation manuelle via Swagger UI complète ces tests automatisés en permettant une exploration visuelle des contrats d'API.

ACC-002 — Authentification JWT. La sécurité de BCaaS repose entièrement sur les JWT : chaque requête vers un endpoint protégé doit porter un token valide, signé avec la clé secrète du serveur, non expiré, et contenant un `tenantId` cohérent avec les données demandées. Les tests de ce critère couvrent le chemin nominale (token valide → accès accordé) et les cas d'attaque les plus courants (token absent, expiré, signature altérée, `tenantId` modifié).

ACC-003 — Cycle CdS → FdS. Ce critère valide le flux métier central de BCaaS. CdS désigne la *Commande de Service* (demande émise par un client) et FdS la *Facture de Service* générée à l'issue de la prestation. Le test d'intégration associé parcourt toutes les transitions de la machine à états et vérifie à chaque étape que : (i) la transition est acceptée ou refusée conformément aux règles métier, (ii) l'état persisté en base est correct, et (iii) les effets de bord attendus (notifications, génération de facture) sont bien déclenchés.

ACC-004 — Génération automatique de la facture. La facture est le document qui matérialise la fin d'une prestation et déclenche le processus de paiement. Sa génération automatique à l'état `fulfilled` est un invariant métier critique : aucune prestation ne doit pouvoir être marquée comme terminée sans qu'une facture valide soit créée. Les tests vérifient également que les montants calculés correspondent aux tarifs du catalogue et que les références croisées (identifiant de la demande, identifiant du tenant, identifiant du prestataire) sont correctement renseignées.

ACC-005 — Isolation multi-tenant. L'isolation multi-tenant est à la fois un critère de sécurité et un critère de conformité. `TenantContextTest` vérifie que le mécanisme de propagation du `tenant_id` par `ThreadLocal` fonctionne correctement dans tous les cas : contexte présent, contexte absent, changement de contexte entre deux requêtes. `SecurityIntegrationTest` complète cette vérification en s'assurant qu'un token portant un `tenantId` A ne permet pas d'accéder aux ressources d'un tenant B.

ACC-006 — Documentation Swagger. La documentation Swagger (générée via SpringDoc/OpenAPI 3) constitue le contrat d'API exposé aux développeurs consommateurs de BCaaS. Sa validité est vérifiée manuellement : chaque endpoint documenté est testé via l'interface Swagger UI pour s'assurer que les paramètres, les corps de requête et les réponses décrits correspondent au comportement réel de l'API.

12.3 Scénario de validation bout-en-bout

12.3.1 Objectif et portée

Le scénario de validation bout-en-bout constitue la démonstration de référence de BCaaS. Son objectif est de valider, en une seule séquence cohérente, l'ensemble des mécanismes fondamentaux du système : authentification JWT, isolation multi-tenant, machine à états de la demande de service, et génération automatique de facture. Ce scénario est conçu pour être reproductible aussi bien par les tests automatisés (`ServiceRequestIntegrationTest`) que par une démonstration manuelle lors d'une soutenance.

12.3.2 Déroulement du scénario nominal

Le scénario se déroule en six étapes séquentielles, chacune produisant un état observable vérifiable :

Étape 1. Inscription et connexion — obtention du JWT.

Un nouvel utilisateur s'inscrit via `POST /api/auth/register` en fournissant ses informations et le nom de son organisation (tenant). BCaaS provisionne automatiquement le tenant, crée le compte utilisateur et retourne un token JWT signé. Ce token contient le `tenantId` du tenant nouvellement créé et est inclus dans l'en-tête `Authorization: Bearer <token>` de toutes les requêtes suivantes.

Vérification : le token est bien formé (trois segments base64 séparés par des points), non expiré, et le `tenantId` est présent dans les *claims*.

Étape 2. Création d'un acteur, d'un portfolio et d'un métier.

L'utilisateur crée un acteur prestataire via `POST /api/actors`, lui associe un portfolio de compétences, puis crée un métier (*business*) via `POST /api/businesses`. Ces entités sont automatiquement associées au tenant extrait du JWT.

Vérification : les réponses HTTP retournent le statut 201, les entités créées sont récupérables via les endpoints `GET` correspondants et portent bien le `tenantId` attendu.

Étape 3. Ajout d'un service au catalogue.

Un service est ajouté au catalogue du métier via `POST /api/service-catalogue`. La définition du service inclut son libellé, sa description, son tarif unitaire et son unité de facturation.

Vérification : le service est visible dans le catalogue du tenant via `GET /api/service-catalogue` et son tarif est correctement persisté.

Étape 4. Émission d'une demande de service (CdS).

Un acteur client émet une demande de service via `POST /api/service-requests`, en référençant le service du catalogue et en précisant les paramètres de la prestation (quantité, date souhaitée, instructions spécifiques). La demande est créée à l'état `PENDING`.

Vérification : la demande est persistée avec l'état `PENDING`, l'identifiant de la demande

est retourné dans la réponse, et la demande est visible dans la liste des demandes du tenant.

Étape 5. Transitions d'état et génération de la facture.

Le prestataire fait progresser la demande à travers les transitions de la machine à états :

- PENDING → ACCEPTED : le prestataire accepte la demande via PATCH `/api/service-requests/{id}/accept`.
- ACCEPTED → IN_PROGRESS : le prestataire démarre la prestation via PATCH `/api/service-requests/{id}/start`.
- IN_PROGRESS → FULFILLED : le prestataire marque la prestation comme terminée via PATCH `/api/service-requests/{id}/fulfill`.

Lors du passage à FULFILLED, BCaaS génère automatiquement une facture (FdS) associée à la demande. Cette génération constitue l'**accusé de réception métier** : elle matérialise la fin de la prestation et déclenche le processus de paiement.

Vérification : après chaque transition, l'état persisté est contrôlé. Après `fulfill`, la facture est récupérable via GET `/api/invoices` et ses montants correspondent au tarif du catalogue multiplié par la quantité de la demande.

Étape 6. Paiement de la facture.

L'acteur client enregistre le paiement de la facture via PATCH `/api/invoices/{id}/pay`. La facture passe à l'état PAID et la demande de service est clôturée.

Vérification : la facture est à l'état PAID, la date de paiement est renseignée, et aucune transition supplémentaire n'est acceptée par la machine à états (idempotence du paiement).

12.3.3 Ce que le scénario valide

Ce parcours en six étapes exerce simultanément les mécanismes les plus critiques de BCaaS :

La machine à états de la demande de service Chaque transition est validée : les transitions autorisées aboutissent, les transitions interdites sont rejetées avec un code HTTP 409. L'ordre des transitions est imposé par la machine à états et ne peut pas être contourné via l'API.

L'isolation multi-tenant Toutes les entités créées (acteurs, métiers, services, demandes, factures) sont automatiquement scopées au tenant du JWT. Un deuxième tenant créé en parallèle ne voit aucune de ces données.

La génération automatique de facture La facture est créée de façon atomique avec la transition `fulfill` : les deux opérations font partie de la même transaction JPA, garantissant qu'il ne peut pas exister de demande FULFILLED sans facture associée.

La cohérence des références croisées La facture référence correctement la demande, le

prestataire, le client et le tenant. Ces références sont vérifiées via des assertions sur les identifiants retournés par les différents endpoints.

12.4 Note sur la couverture de code et perspectives

12.4.1 État actuel

Aucun outil de mesure de couverture de code (*code coverage*) n'est actuellement intégré au pipeline de build de BCaaS. Les 152 tests constituent néanmoins une base de non-régression substantielle qui couvre :

- l'ensemble des chemins nominaux de tous les services métier ;
- les principaux cas d'erreur et cas limites identifiés durant le développement ;
- le cycle de vie complet de la demande de service, de l'émission au paiement ;
- les mécanismes de sécurité (authentification, autorisation, isolation tenant).

En l'absence de mesure formelle, il est impossible d'affirmer un taux de couverture précis. Cependant, la couverture fonctionnelle — c'est-à-dire la proportion des fonctionnalités métier couvertes par au moins un test — est estimée à un niveau satisfaisant pour un MVP.

12.4.2 Perspectives d'amélioration

Plusieurs améliorations sont identifiées pour les versions futures de BCaaS (voir chapitre 14) :

Intégration de JaCoCo JaCoCo (*Java Code Coverage Library*) s'intègre nativement avec Maven et Gradle. Son ajout permettrait de générer un rapport de couverture à chaque build, d'identifier les branches non testées et de définir des seuils minimaux de couverture (par exemple, 80 % de couverture de branches) en dessous desquels le build échoue.

Tests de mutation Les tests de mutation (avec des outils comme PIT/Pitest) permettent d'évaluer la *qualité* des tests plutôt que leur simple existence : ils introduisent délibérément des fautes dans le code (mutations) et vérifient que les tests les détectent. Un test qui passe même en présence d'une mutation est un test insuffisant.

Tests de charge BCaaS étant un service multi-tenant destiné à héberger plusieurs organisations simultanément, des tests de charge avec des outils comme Gatling ou JMeter permettraient de valider les performances sous des montées en charge réalistes et d'identifier les goulots d'étranglement avant la mise en production.

Tests de contrat (Consumer-Driven Contract Testing) Si BCaaS est consommé par plusieurs applications clientes, les tests de contrat (avec Pact, par exemple) garantiraient que les évolutions de l'API ne cassent pas silencieusement les consommateurs existants.

Synthèse

Ce chapitre a présenté la démarche complète de validation de BCaaS. La stratégie de test repose sur une pyramide à trois niveaux — tests unitaires, tests de contrôleurs et tests d'intégration —

mobilisant 16 classes et 152 méthodes sur un environnement H2 en mémoire parfaitement isolé. Les six critères d'acceptation formels garantissent que les fonctionnalités les plus critiques — sécurité JWT, isolation multi-tenant, cycle CdS → FdS — sont vérifiées de manière systématique et reproductible.

Le scénario de validation bout-en-bout constitue la démonstration de référence du système : en six étapes cohérentes, il exerce simultanément la machine à états, l'isolation tenant et la génération automatique de facture, offrant une preuve d'intégration concrète et démontrable lors de la soutenance.

L'absence de mesure formelle de couverture constitue la principale limite reconnue de ce dispositif ; son adressage, via l'intégration de JaCoCo et de tests de mutation, figure parmi les priorités du plan d'évolution présenté au chapitre 14.

CHAPITRE 13

Perspectives d'amélioration

Le MVP livré pose des fondations solides ; il ouvre néanmoins de nombreuses pistes d'amélioration, dont plusieurs correspondent aux contraintes technologiques avancées du sujet. Cette feuille de route est organisée par horizon.

13.1 Court terme : finalisation des briques posées

- **Câblage de l'événementiel Kafka.** L'infrastructure (producteur configuré, topic `bizcore.events`, consommateur de notifications) est en place ; il reste à publier effectivement les événements de domaine (changement de statut d'une demande, génération de facture) pour rendre la chaîne pleinement *event-driven*.
- **Activation du cache Redis.** Le gestionnaire de cache et les TTL sont définis ; l'ajout d'annotations `@Cacheable` / `@CacheEvict` sur les lectures fréquentes (règles métier, configuration tenant, catalogue) activerait le cache déjà préfixé par tenant.
- **Limitation de débit (rate limiting).** La dépendance `Bucket4j` est présente ; son intégration permettrait d'appliquer des quotas par tenant, prolongeant l'analogie de la bande passante et de la QoS.
- **Mesure de couverture de tests.** L'ajout de `JaCoCo` fournirait un indicateur chiffré de couverture.
- **Sur le plan de l'architecture:** il s'agirait d'étendre la couverture des cent cinquante-deux tests automatisés afin d'inclure des scénarios de charge et des tests d'intégration multitenant plus poussés, garantissant la robustesse de l'isolation par `tenantId` dans des conditions de concurrence élevée
- **La documentation OpenAPI:** pourrait également être enrichie d'exemples d'utilisation et de scénarios d'erreur, facilitant l'adoption de l'API par de nouveaux développeurs
- **le service d'analytique et la piste d'audit:** pourraient être complétés par des tableaux de bord de supervision en temps réel, exploitant les données déjà collectées pour offrir une visibilité immédiate sur l'activité de chaque tenant

13.2 Moyen terme : robustesse et qualité de service

- **Isolation multi-tenant renforcée.** Passer de la discipline applicative à une barrière systématique au moyen d'un filtre Hibernate `@Filter` ou de la *Row-Level Security* PostgreSQL, voire d'une isolation par schéma dédié pour les tenants premium.

- **QoS différenciée.** Introduire des niveaux de service inspirés de DiffServ (Bronze, Silver, Gold, Platinum) combinant priorité, latence cible et stratégie d'isolation.
- **Observabilité complète.** Exposer des métriques via Prometheus et un tableau de bord Grafana, et mettre en place des logs structurés agrégés.
- **Le modèle générique** pourrait être étendu pour couvrir de nouveaux domaines métier au-delà du cas d'usage initial, validant ainsi la généricité réelle du Business Core sur des secteurs d'activité variés (santé, transport, logistique)
- **L'authentification JWT et le contrôle d'accès basé sur les rôles :** pourraient évoluer vers une gestion plus fine des permissions, avec la possibilité pour chaque tenant de définir ses propres rôles et politiques d'autorisation sans intervention sur le code du système

13.3 Long terme : montée en charge et nouvelles contraintes

- **Architecture réactive.** Migration vers Spring WebFlux + R2DBC pour un modèle non bloquant, mieux adapté à une forte concurrence.
- **Géo-référencement natif.** Intégration de PostGIS et d'attributs de localisation pour les recherches géo-spatiales (*geolocation aware*).
- **Stockage d'objets.** Adoption de MinIO pour les médias, au lieu de la simple URL actuellement stockée.
- **Sécurité renforcée.** Double authentification (2FA) et migration vers OAuth 2.0.
- **Front offline-first et international.** Transformation du frontend en PWA installable, mode hors-ligne, et internationalisation (i18n) web et mobile.
- **Application mobile native.** Client React Native partageant l'API.
- **Microservices.** Extraction progressive des modules en services déployés indépendamment, le découpage actuel ayant été pensé pour cette évolution.
- **Marketplace d'instances.** À terme, permettre à des entreprises de déployer leur propre configuration BCaaS sans développement, comme socle d'un écosystème d'instances métier.
- **La grille de lecture protocolaire en cinq couches :** , actuellement formalisée et reliée au modèle OSI, pourrait servir de fondement à une véritable spécification ouverte du Business Core as a Service, documentée et partagée, permettant à d'autres équipes de contribuer à son évolution ou de l'adopter comme standard interne

CHAPITRE 14

Conclusion générale

14.1 Bilan

Ce projet a démontré qu’une analogie rigoureuse entre l’ingénierie des protocoles réseaux et la conception d’une plateforme métier n’est pas une simple métaphore pédagogique : elle constitue une véritable méthode de conception. Les concepts de couches, d’encapsulation, de routage, d’adressage, de qualité de service et de séparation des plans de contrôle et de données ont guidé chaque décision architecturale de BCaaS.

Les réalisations principales sont :

- une grille de lecture protocolaire en cinq couches, formellement reliée au modèle OSI ;
- un modèle de données générique de quinze entités, configurable par tenant et implémenté sur PostgreSQL avec Liquibase ;
- une API REST d’environ quatre-vingt-cinq endpoints, documentée par OpenAPI et validée par cent cinquante-deux tests automatisés ;
- une authentification JWT multi-tenant avec contrôle d’accès basé sur les rôles, et une isolation par colonne `tenant_id` pilotée par un contexte `ThreadLocal` ;
- un protocole d’échange formalisé pour un cas métier concret ;
- une piste d’audit et un service d’analytique pour la traçabilité ;
- un frontend Next.js (site vitrine et tableau de bord), l’ensemble étant déployé sur le cloud (Render et Vercel).

14.2 Limites

Le périmètre livré est celui d’un MVP. Plusieurs briques avancées du sujet sont posées mais non pleinement câblées (événementiel Kafka, cache Redis par annotations, limitation de débit) ou restent à venir (réactif, géo-référencement, stockage objet, 2FA, PWA/offline, i18n, mobile natif).

Par ailleurs, la couverture des tests automatisés, bien que substantielle, reste centrée sur la validation fonctionnelle de l’API et n’inclut pas encore de tests de charge approfondis permettant d’évaluer le comportement du système face à un grand nombre de tenants simultanés. Enfin, le contrôle d’accès basé sur les rôles, bien qu’opérationnel, demeure relativement statique : chaque tenant ne dispose pas encore de la capacité à définir ses propres rôles et politiques d’autorisation de façon autonome.

Ces limites, assumées et documentées, font l’objet de la feuille de route du chapitre 13.

14.3 Apports personnels

Ce projet a permis d’acquérir des compétences concrètes en architecture logicielle distribuée, en conception d’API REST professionnelle, en sécurité applicative (JWT, RBAC), en isolation multi-tenant et en modélisation de systèmes génériques. Au-delà des technologies, la démarche de *pensée protocolaire* — définir les contrats avant le code, modéliser les échanges comme des messages — constitue un réflexe d’ingénieur directement transposable à tout projet de système d’information.

Ce travail a également renforcé notre capacité à raisonner en termes de systèmes complets plutôt qu’en fonctionnalités isolées, en nous habituant à anticiper, dès la conception, des préoccupations transverses telles que la sécurité, la traçabilité et l’isolation des données, plutôt que de les traiter comme des ajouts tardifs. Cette approche constitue, selon nous, un acquis précieux pour notre future pratique en tant qu’ingénieurs en informatique.

Bibliographie

- [1] Pr. Dr Ing. Thomas Djotio Ndié, *Cours d'Introduction aux Réseaux Locaux*, 3GI, ENSPY, 2025-2026.
- [2] ISO/IEC 7498-1, *Information technology — Open Systems Interconnection — Basic Reference Model*, ISO, 1994.
- [3] A. Cockburn, *Hexagonal Architecture (Ports and Adapters)*, 2005.
- [4] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, 2003.
- [5] M. Jones, J. Bradley, N. Sakimura, *RFC 7519 — JSON Web Token (JWT)*, IETF, 2015.
- [6] <https://www.access-it.fr/actualite/quest-ce-quune-application-metier/>
- [7] AWS Whitepaper, *SaaS Tenant Isolation Strategies: Isolating Resources in a Multi-Tenant Environment*, <https://docs.aws.amazon.com/pdfs/whitepapers/latest/saas-tenant-isolation-strategies/saas-tenant-isolation-strategies.pdf>,
- [8] Peter Mell Timothy Grance *The NIST Definition of Cloud Computing*, <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-145.pdf>,
- [9] Architecture hexagonale, <https://thetribe.io/glossaire-architecture-hexagonale-clean-archi>
- [10] Spring Boot Reference Documentation, <https://docs.spring.io/spring-boot/>,
- [11] SpringDoc OpenAPI Documentation, <https://springdoc.org>,
- [12] Next.js Documentation, <https://nextjs.org/docs>,
- [13] GeeksforGeeks, *Single-Tenant vs Multi-Tenant Architecture*, <https://www.geeksforgeeks.org/system-design/single-tenant-vs-multi-tenant-architecture/>,
- [14] okta *Multi tenancy cloud* , <https://www.okta.com/fr-fr/identity-101/multi-tenancy-cloud/>

Glossaire

BaaS (Business as a Service)

Modèle fournissant des capacités métier génériques via une API standardisée.

Tenant

Organisation cliente disposant d'une isolation logique de ses données et configurations ; analogue d'un VLAN.

Multi-tenant

Architecture servant plusieurs tenants sur une infrastructure partagée, avec isolation de leurs données.

Actor

Rôle joué par un utilisateur dans le système : consommateur (CdS) et/ou fournisseur (FdS) de service.

CdS / FdS

Consommateur de Service (émetteur de la requête) / Fournisseur de Service (récepteur et exécutant).

Business

Métier ou profession référencé dans le système (domaine, règles, services, ressources).

BusinessRule

Règle métier sous forme de paire clé-valeur ; analogue du protocole.

ServiceRequest

Demande de service entre un CdS et un FdS ; analogue d'un message réseau, porteuse d'un `traceId`.

Invoice

Facture générée automatiquement à la finalisation ; analogue d'un accusé de réception (ACK).

Portfolio

Ensemble des métiers maîtrisés par un acteur (relation N:M avec Business).

FSM

Machine à états finis décrivant le cycle de vie d'une entité (ServiceRequest, Invoice).

JWT

Jeton signé portant l'identité, les rôles et le `tenantId` de l'utilisateur authentifié.

RBAC

Contrôle d'accès basé sur les rôles.

traceId / correlationId

Identifiants de flux permettant de tracer une transaction de bout en bout.

DTO

Objet de transfert de données découplant la représentation exposée des entités persistées.

Soft delete

Suppression logique préservant les données en base (`isActive = false`).

Control plane / Data plane

Plan de configuration (administration) / plan d'exécution (transactions), par analogie avec le SDN.

ANNEXE A

Annexes

A.1 Liste des endpoints REST

L'API expose environ 85 endpoints répartis sur 17 contrôleurs. Le tableau A.1 en donne une vue synthétique.

Table A.1: *Synthèse des endpoints REST*

Ressource	Endpoints
/api/auth	POST /register, POST /login, PATCH /users/{id}/roles, DELETE /users/{id}/roles
/api/tenants	GET, GET /{id}, POST /register, PUT /{id}, DELETE /{id}
/api/users	GET, GET /{id}, GET /email/{email}
/api/actors	GET, GET /{id}, GET /user/{userId}, GET /role/{role}, POST /user/{userId}, DELETE /{id}
/api/businesses	GET, POST, GET /{id}, GET /domain/{domain}, GET /search, PUT /{id}, DELETE /{id}
/api/business-rules	GET, GET /{id}, GET /business/{businessId}, POST /business/{businessId}, PUT /{id}, DELETE /{id}
/api/service-catalogues	GET, GET /{id}, GET /business/{businessId}, GET /available, GET /search, POST /business/{businessId}, PUT /{id}, DELETE /{id}
/api/service-requests	GET, GET /{id}, GET /consumer/{id}, GET /provider/{id}, GET /status/{status}, POST /consumer/{id}/provider/{id}/catalogue/{id}, PATCH /{id}/accept, /start, /fulfill, /cancel, DELETE /{id}
/api/invoices	GET, GET /{id}, GET /status/{status}, GET /service-request/{id}, POST /service-request/{id}, PATCH /{id}/pay, /cancel, DELETE /{id}
/.../messages	GET, POST, GET /unread-count, PATCH /read-all, DELETE /{messageId}
/api/notifications	GET, GET /unread-count, PATCH /{id}/read, PATCH /read-all, DELETE /{id}
/api/portfolios	GET, GET /{id}, GET /actor/{id}, POST /actor/{id}, PATCH /{id}/businesses/{bid}, DELETE /{id}/businesses/{bid}, DELETE /{id}
/api/resources	GET, GET /{id}, GET /business/{id}, POST /business/{id}, PUT /{id}, DELETE /{id}
/api/media	GET, GET /{id}, GET /business/{id}, GET /type/{type}, POST /business/{id}, PUT /{id}, DELETE /{id}
/api/currencies	GET, GET /validate/{code}
/api/analytics	GET /tenants/{tenantId}/kpis

Ressource	Endpoints
/api/audit	GET /service-requests/{id}, GET /actors/{id}, GET /trace/{traceId}, GET /tenants/{id}

A.2 Exemple de log structuré

```

1 {
2   "timestamp": "2025-05-04T10:05:23Z",
3   "level": "INFO",
4   "tenant_id": "550e8400-e29b-41d4-a716-446655440000",
5   "trace_id": "7e9a8b7c-6d5e-4f3a-2b1c-9d8e7f6a5b4c",
6   "actor_id": "actor_123",
7   "service": "BusinessService",
8   "operation": "createBusiness",
9   "duration_ms": 145,
10  "http_status": 201
11 }
```

Listing A.1: Log structuré au format JSON (proposition)

A.3 Capture Swagger UI

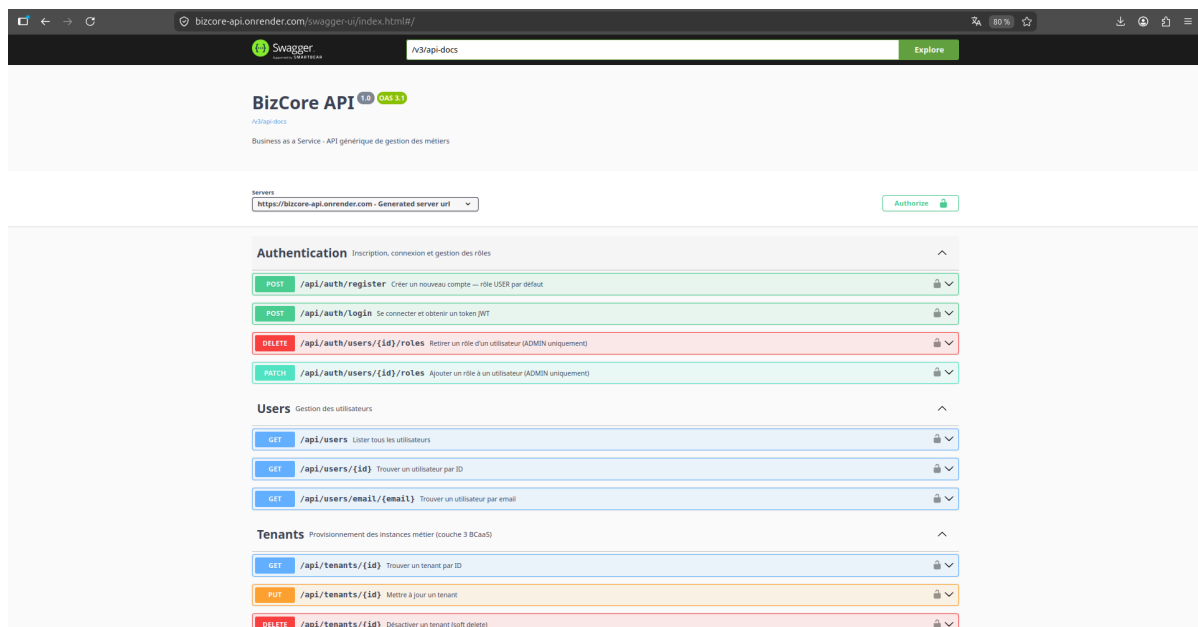


Figure A.1: Documentation interactive Swagger UI

A.4 Extraits de code significatifs

```
1 public final class TenantContext {  
2     private static final ThreadLocal<UUID> CURRENT_TENANT = new ThreadLocal  
        <>();  
3     public static void setTenantId(UUID tenantId) { CURRENT_TENANT.set(  
        tenantId); }  
4     public static UUID getTenantId() { return CURRENT_TENANT.get(); }  
5     public static void clear() { CURRENT_TENANT.remove(); }  
6 }
```

Listing A.2: *TenantContext (ThreadLocal)*

```
1 UUID tenantId = TenantContext.getTenantId();  
2 if (tenantId != null) {  
3     return serviceRequestRepository.findAllByTenantId(tenantId, pageable);  
4 }
```

Listing A.3: *Lecture tenant-aware dans un service*